

New Data Structure: Disjoint Sets

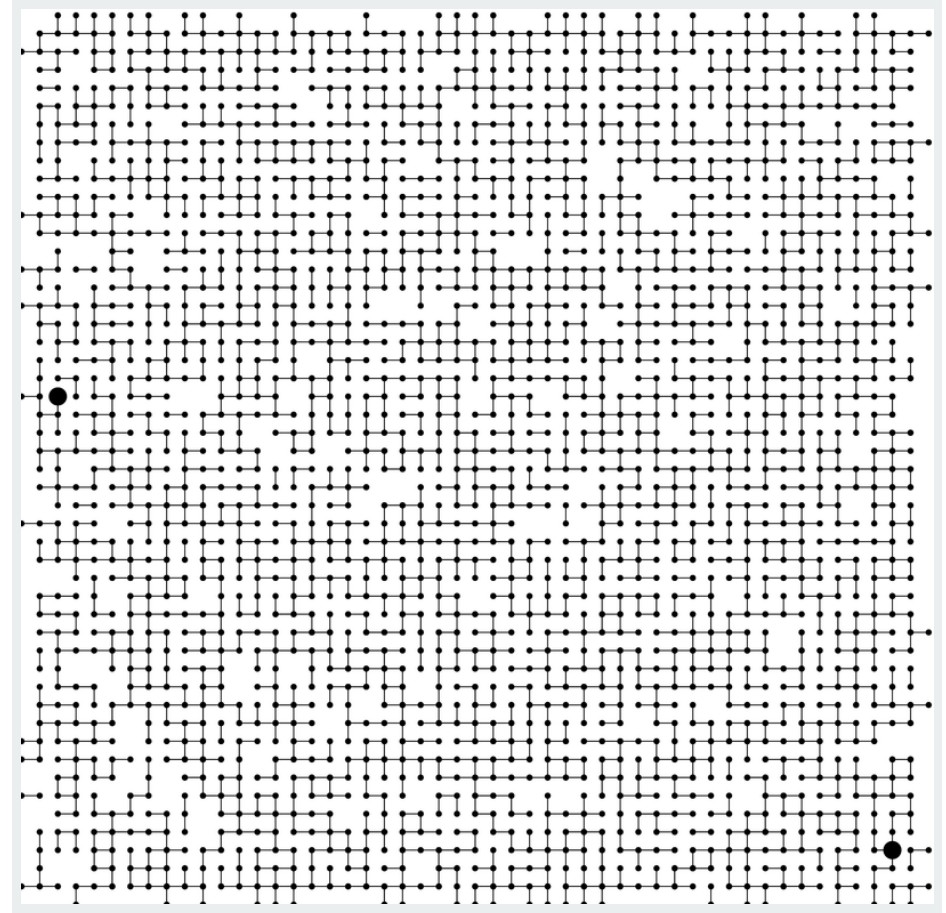
- Problem of Interest: keep track of dynamic relations
 - e.g. who is connected to whom
 - Who belongs to same club
 - Which computers are in same network
 - Which web pages link on the Internet
- Abstraction
 - Have a set of objects
 - Have relations between objects:
 - symmetric, $a \sim b$, transitive $a \sim b, b \sim c \longrightarrow a \sim c$
 - Want to keep track of subsets of objects that are related as relations are added

Disjoint Sets

- Operations
 - Find(a): Find the subset that contains element a
 - Union(a,b): Add a relation between two elements a, b which merges the subsets containing a, b
 - Often known as Union-Find problem
- Would like to do Unions and Find in $O(1)$
 - Seems really hard...
 - We'll come close with a disjoint set data structure

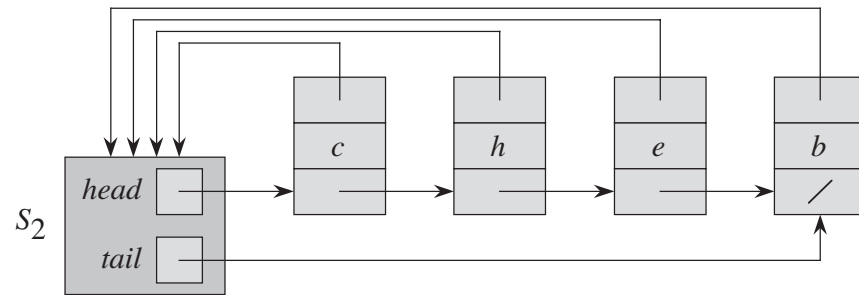
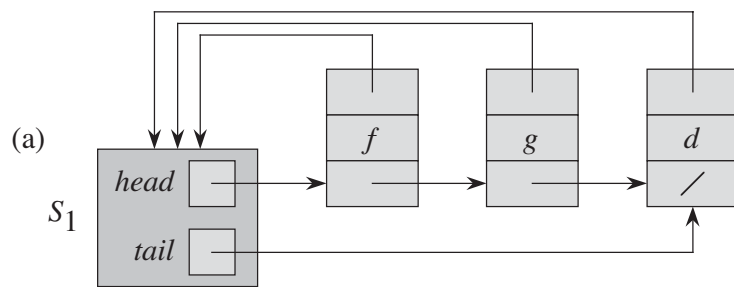
Disjoint Sets in Action

- Application: Find path in graph
 - Do start, end belong to same subset
 - Relation: a, b related if edge exists between them
- Solve: Do start and end belong to same connected set?



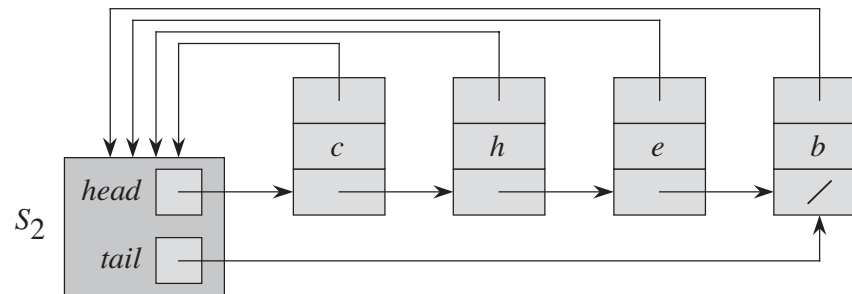
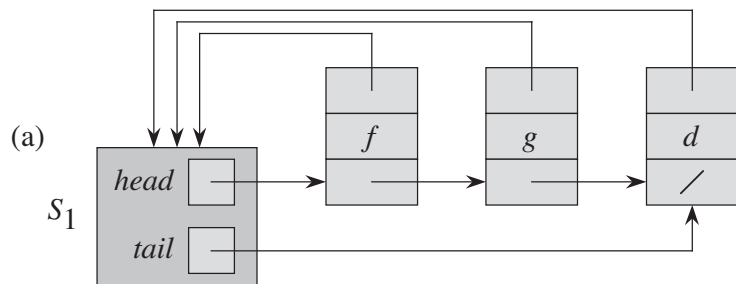
Disjoint Sets

- Possible solution
 - Represent subsets as linked lists, with a head object as its identifier
 - All list elements point to head, and to next in list
 - Head points to first child and to last child



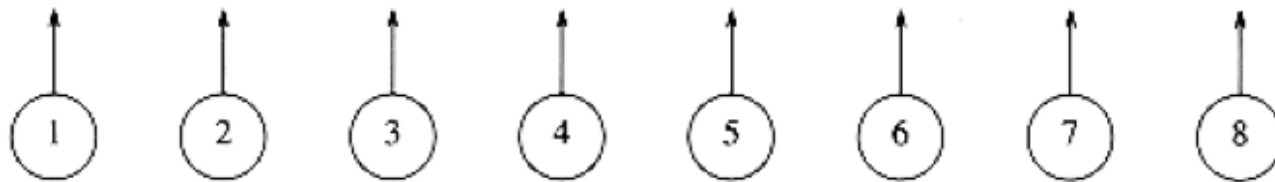
Disjoint Sets

- Possible solution Complexity
 - Find(a) is $O(1)$ using pointers to head
 - Union(a,b): Slow, needs to update pointers to new head
 - $O(n)$ worst case

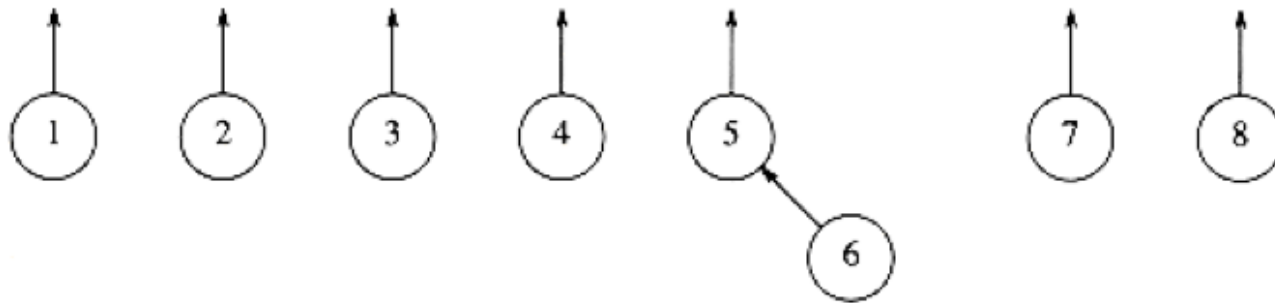


Disjoint Sets

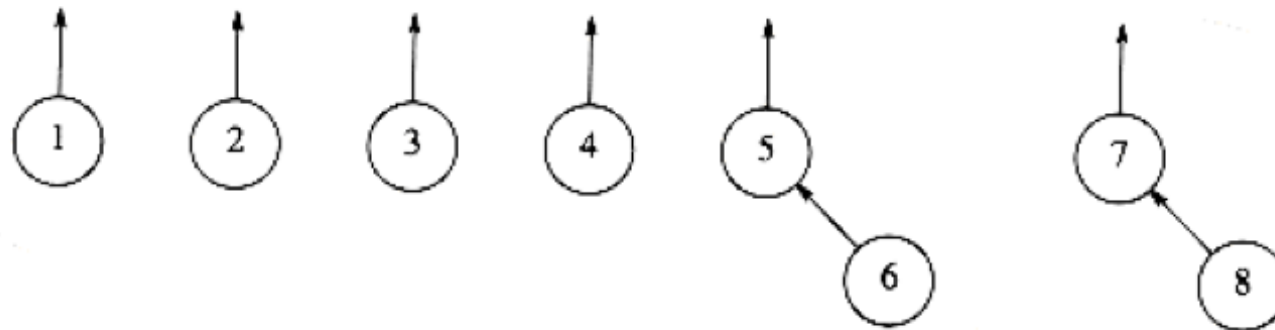
- Better Solution
 - Keep track of subsets as trees, children point to parent, root is subset label
 - Find: get root label of tree, moving up the tree
 - Union: if new relation $a \sim b$, Find(a) and Find(b)
 - If a, b are in different trees, merge trees
 - How to merge: Keep track of **height of trees**
 - Merge shorter tree as direct child taller tree



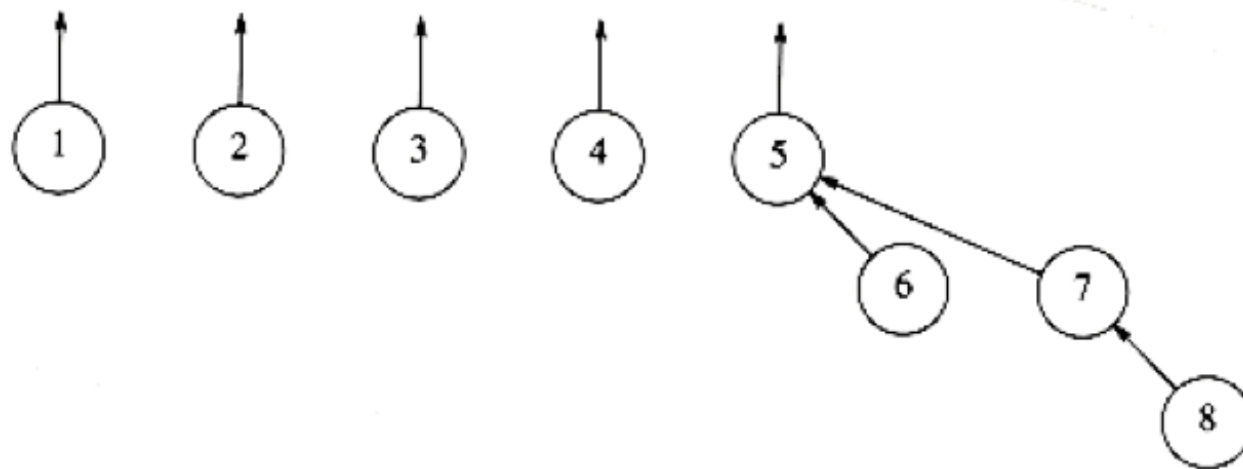
Initial forest:
8 disjoint trees



Add relation: Union(5,6)



Add relation: Union(7,8)

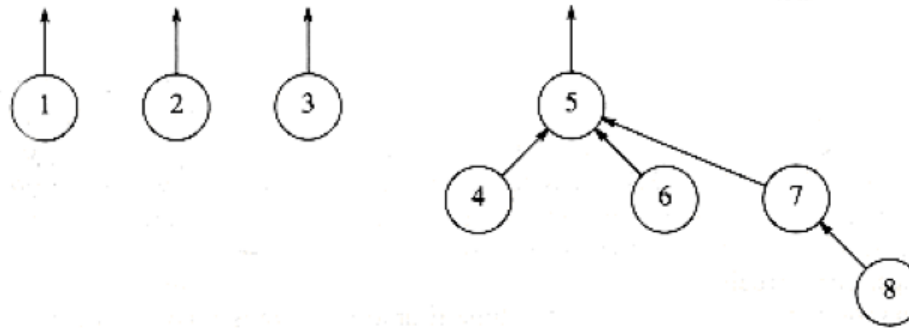


Add relation: Union(6,8)

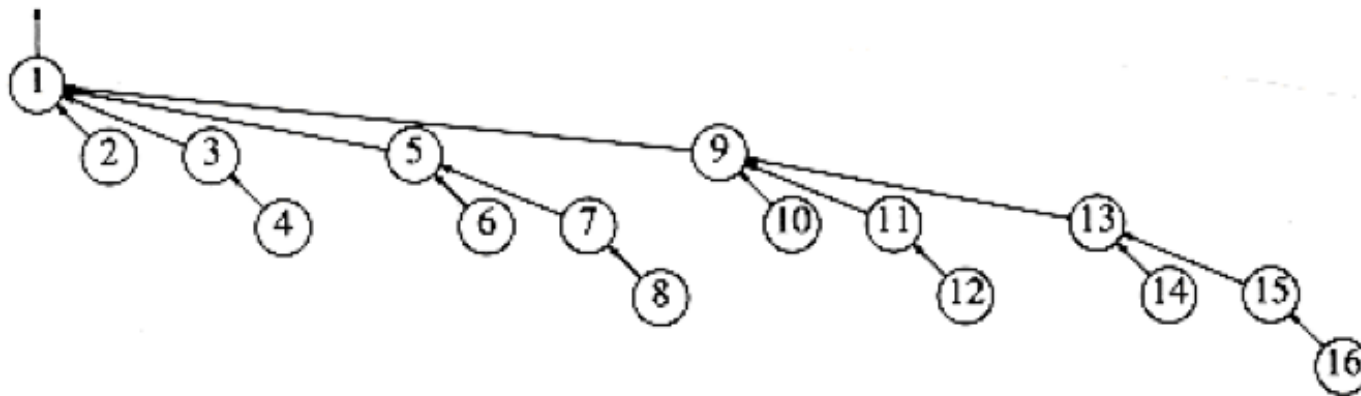
Simple Data Structure: Each node points to its parent
(roots point to negative value of their height)

Parent	0	0	0	0	-2	5	5	7
Node	1	2	3	4	5	6	7	8

Union (4,7)



Short tree links to tall:
Union by height!



Binomial tree!

Worst case of union-by-height:

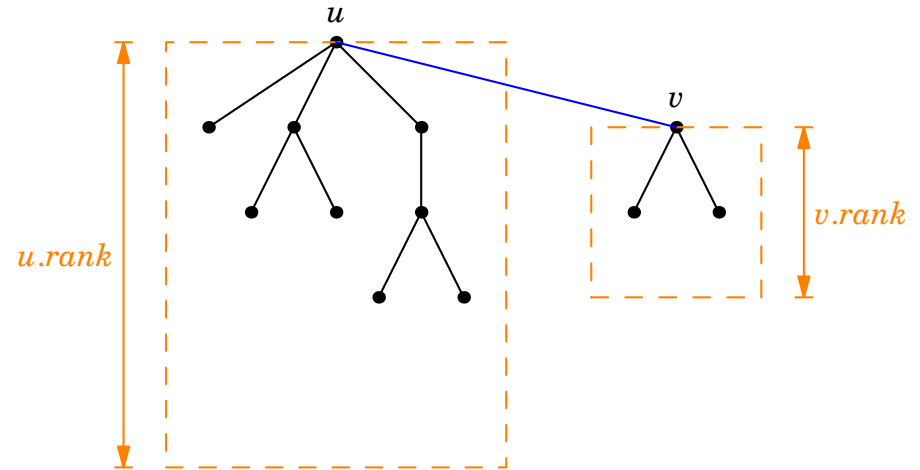
For N keys in tree, largest depth is $O(\log(N))$

Disjoint Sets

- Complexity
 - Height update on merge:
 - h_1 is height of taller tree, h_2 is height of shorter tree
 - If $h_1 > h_2$, height of h_1 does not change
 - If $h_1 = h_2$, h_1 increases by 1
 - Find complexity: time to find root of tree = height of tree: $O(\log(n))$
 - Merge complexity: compare heights, point shorter root to taller one: $\Theta(1)$
 - Worst case complexity of union-find: $O(\log(n))$
(too slow!)

Updating Ranks

- **Rank** is an **upper bound** on height
 - Height is hard to keep track of exactly because of path compression
- How is rank assigned?
 - Rank of single nodes = 0
 - If 2 trees of equal ranks are merged, $\text{rank}(\text{new root}) = \text{rank}(\text{trees}) + 1$
 - Without path compression, this makes $\text{rank} = \text{height}$
 - After path compression, rank is upper bound to height
- **Ranks only change for root when adding children**
 - Don't update ranks of non-roots because only roots matter!

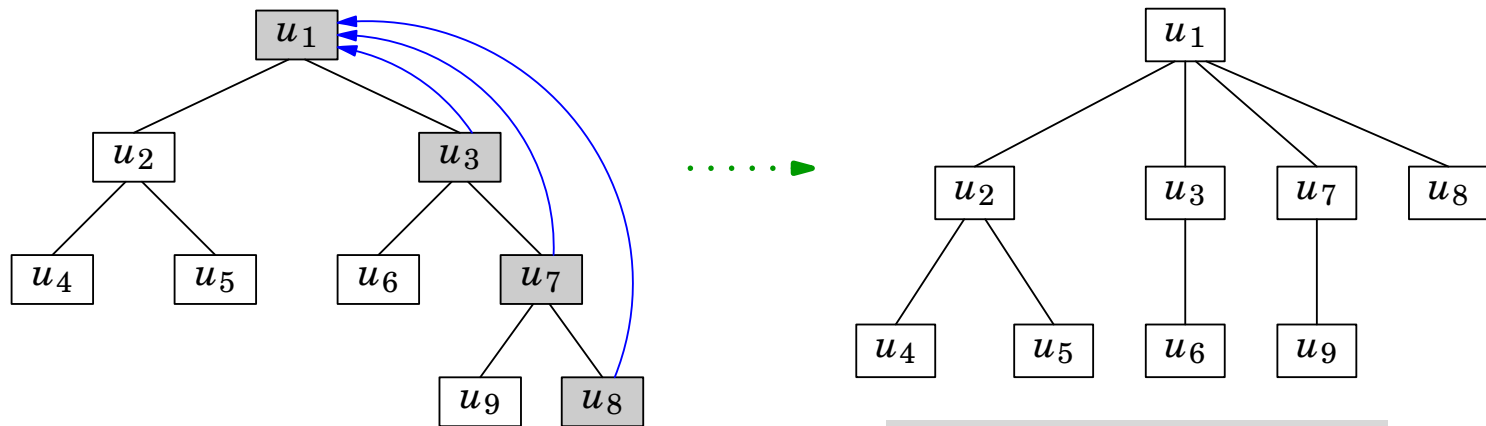


Algorithm: UNION($\tilde{u}, \tilde{v}$)

```
1  $u \leftarrow \text{FIND-SET}(\tilde{u})$ 
2  $v \leftarrow \text{FIND-SET}(\tilde{v})$ 
3 if  $u.rank = v.rank$  then
4      $u.rank \leftarrow u.rank + 1$ 
5      $v.parent \leftarrow u$ 
6 else if  $u.rank > v.rank$  then
7      $v.parent \leftarrow u$ 
8 else
9      $u.parent \leftarrow v$ 
```

More on Rank and Path Compression

- Path compression illustration: Find u_8



- Rank does not change!
 - But height dropped by 1

Algorithm: FIND-SET(u)

```
1  $A \leftarrow \emptyset$ 
2  $n \leftarrow u$ 
3 while  $n$  is not the root do
4    $A \leftarrow A \cup \{n\}$ 
5    $n \leftarrow n.parent$ 
6 for each  $x \in A$  do
7    $x.parent \leftarrow n$ 
8 return  $n$ 
```

EC504 ALGORITHMS AND DATA STRUCTURES
SPRING 2025 MONDAY & WEDNESDAY
2:30 PM - 4:15 PM

Prof: David Castañón, dac@bu.edu

Teaching Assts:

Seyda Guzelhan : seyda@bu.edu

Disjoint Sets

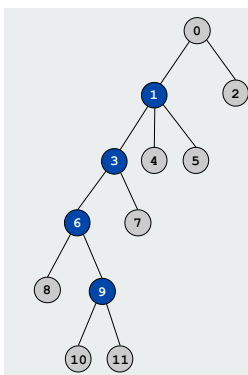
- Operations
 - Find(a): Find the subset that contains element a
 - Union(a,b): Add a relation between two elements a, b which merges the subsets containing a, b
 - Often known as Union-Find problem
- Would like to do Unions and Find in $O(1)$
 - Seems really hard...
 - We'll come close with a disjoint set data structure

Disjoint Sets

- Better Solution
 - Keep track of subsets as trees, children point to parent, root is subset label
 - Find: get root label of tree, moving up the tree
 - Union: if new relation $a \sim b$, Find(a) and Find(b)
 - If a, b are in different trees, merge trees
 - How to merge: Keep track of **height of trees**
 - Merge shorter tree as direct child taller tree

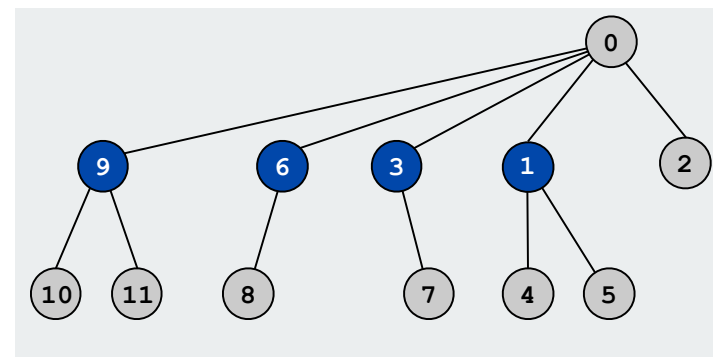
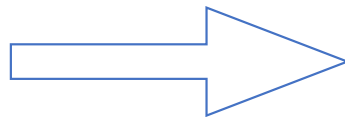
Improved Disjoint Sets Algorithm

- Approach: restructure trees when doing finds to reduce height!
 - Find: get root label of tree, moving up the tree
 - Restructure tree by **making all ancestors have root as parent**
 - Union: if new relation $a \sim b$, Find(a) and Find(b). If a, b are in different trees, merge trees
 - Merge: roots have **rank**s. Merge root so smaller rank is child of larger rank one; if same rank, increase new root rank by 1



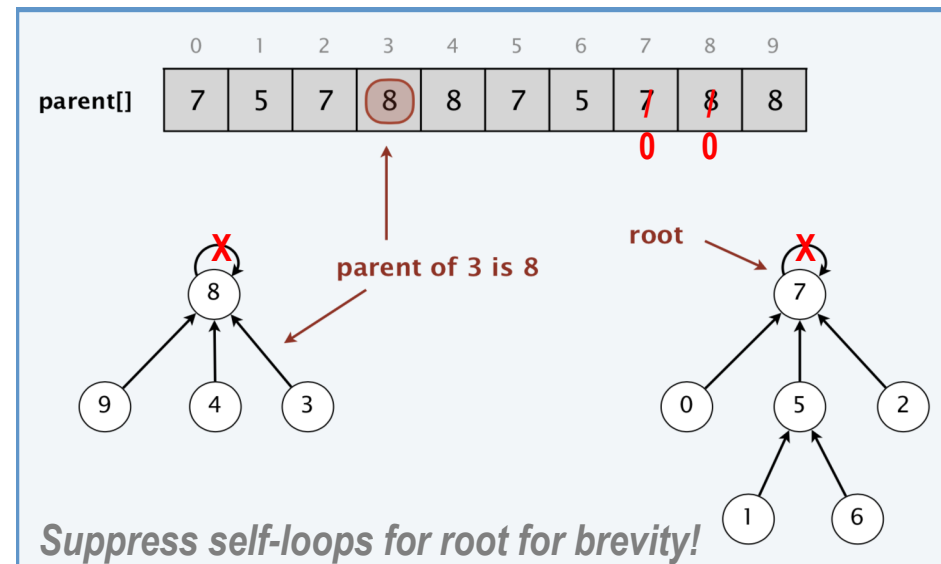
Path Compression

Find 9



Representation of Disjoint Set Operations

- **Array:** Objects indexed in the array, with value pointing to parent location
 - Roots point to self-location
 - Keep track of separate array for ranks
 - Alternative: For roots, use negative of rank as entry (don't need ranks of non-roots)
- How is rank assigned?
 - Rank of single nodes = 0
 - If 2 trees of equal ranks are merged, $\text{rank}(\text{new root}) = \text{rank}(\text{trees}) + 1$
 - Without path compression, this makes $\text{rank} = \text{height}$
 - After path compression, rank is upper bound to height



Disjoint Sets

- **Theorem:** Starting from an empty data structure, any sequence of M union and find operations on N objects takes $O(N + M \log^*(N))$ time
 - $\log^*(N)$: Ackerman function
 - $A(i) = 2^{A(i-1)}$; $A(0) = 1$
 - $A(1) = 2$, $A(2) = 4$, $A(3) = 16$, $A(4) = 2^{16} = 65536$, $A(5) = \dots$,
 - $A(6) = \text{VERY VERY VERY BIG!}$
 - Inverse Ackerman: $i = \log^*(N)$
 - $\log^*(N) = \text{min number times you take } \log_2(\log_2(\dots)) \text{ to get smaller than or equal to } 1$
 - For all practical purposes, it is $O(1)$ (Ch. 21, CLRS)
 - Disjoint set complexity per union-find: almost $O(1)$

Outline of Proof of Complexity

- Properties of rank (**without path compression**): Let $p[x]$ be $\text{parent}[x]$
 - x is not a root node, then $\text{rank}(x) < \text{rank}(p[x])$
 - Rank k root created by linking two roots of rank $k - 1$
 - x is not a root node, then $\text{rank}(x)$ never changes
 - When $p[x]$ changes, then $\text{rank}(p[x])$ strictly increases
 - $p[x]$ only changes for linking to root
 - When $p[x]$ changes, $\text{rank}(x) < \text{rank}(p[x])$
 - $p[x]$ only changes for roots, when one root goes under other

Algorithm: $\text{UNION}(\tilde{u}, \tilde{v})$

```
1  $u \leftarrow \text{FIND-SET}(\tilde{u})$ 
2  $v \leftarrow \text{FIND-SET}(\tilde{v})$ 
3 if  $u.\text{rank} = v.\text{rank}$  then
4      $u.\text{rank} \leftarrow u.\text{rank} + 1$ 
5      $v.\text{parent} \leftarrow u$ 
6 else if  $u.\text{rank} > v.\text{rank}$  then
7      $v.\text{parent} \leftarrow u$ 
8 else
9      $u.\text{parent} \leftarrow v$ 
```

Outline of Proof of Complexity - 2

- Properties of rank (without path compression): Let $p[x]$ be $\text{parent}[x]$
 - Root of rank k has $\geq 2^k$ elements in its tree
 - Induction: true for $k = 0$
 - Assume true for $k-1$. Root gets rank k by merging two roots of rank $k-1$. Thus, no. elements $\geq 2^{k-1} + 2^{k-1} \geq 2^k$
 - Highest rank of root $\leq \lfloor \log_2(n) \rfloor$
 - For any integer k , there are $\leq n/2^k$ nodes with rank k
 - Non-root nodes with rank k were roots with rank k at some point
 - Every rank k node has at least 2^k elements,

Algorithm: UNION($\tilde{u}, \tilde{v}$)

```
1  $u \leftarrow \text{FIND-SET}(\tilde{u})$ 
2  $v \leftarrow \text{FIND-SET}(\tilde{v})$ 
3 if  $u.\text{rank} = v.\text{rank}$  then
4      $u.\text{rank} \leftarrow u.\text{rank} + 1$ 
5      $v.\text{parent} \leftarrow u$ 
6 else if  $u.\text{rank} > v.\text{rank}$  then
7      $v.\text{parent} \leftarrow u$ 
8 else
9      $u.\text{parent} \leftarrow v$ 
```

Outline of Proof of Complexity - 3

- Path compression: Let $p[x]$ be $\text{parent}[x]$
 - Key property: roots, node ranks, tree elements don't change!
All the properties of union by rank stay true
parent pointers updated to root means $\text{rank}(p[x]) > \text{rank}(x)$ still

- Iterated logarithm function

$$\log^*(n) = \begin{cases} 0 & n \leq 1 \\ 1 + \log^*(\log_2(n)) & \text{otherwise} \end{cases}$$

n	$\lg^* n$
1	0
2	1
[3, 4]	2
[5, 16]	3
[17, 65536]	4
[65537, 2 ⁶⁵⁵³⁶]	5

Outline of Proof of Complexity - 4

- Analysis: Buckets of elements
 - Divide elements by non-zero ranks into the following groups
 - $\{1\}, \{2, 3\}, \{4, \dots, 15\}, \dots, \{B, 2^B - 1\}, \{2^B, 2^{2^B} - 1\}$
 - Total number of buckets $\leq \log^*(n)$
 - Max elements in bucket $\{B, 2^B - 1\}$: $\leq n/2^B$
 - Every node has rank in the first $\log^*(n)$ groups
- Creative accounting
 - Every node receives credits the moment it ceases to be a root:
 - If node rank is in $\{B, 2^B - 1\}$, get 2^B credits
 - Number of credits disbursed to all nodes $\leq n \log_2^*(n)$

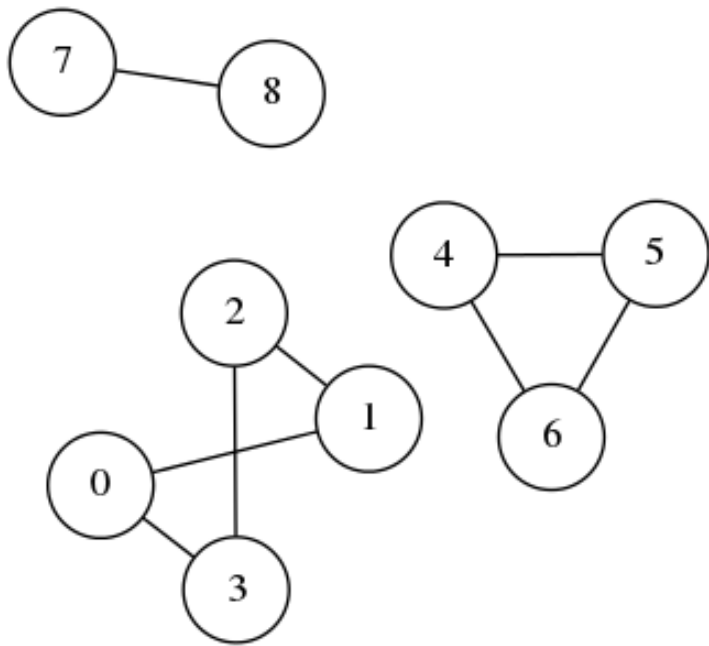
Outline of Proof of Complexity - 5

- Creative accounting
 - Every node receives credits the moment it ceases to be a root:
 - If node rank is in $\{B, 2^B - 1\}$, get 2^B credits
 - Number of credits disbursed to all nodes $\leq n \log^*(n)$
 - Max nodes in bucket $\{B, 2^B - 1\}$: $\leq n/2^B$, so max of n credits in bucket
 - Run time of Find(x): bounded by number of parent pointers to root
 - Rank increases strictly as you search up the tree
 - 3 Cases:
 - Case 0: parent[x] is a root $\rightarrow$ happens for 1 link per find
 - Case 1: rank(parent[x]) is in higher bucket than rank[x]
 - Case 2: rank(parent[x]) is in same bucket as rank[x]

Outline of Proof of Complexity - 6

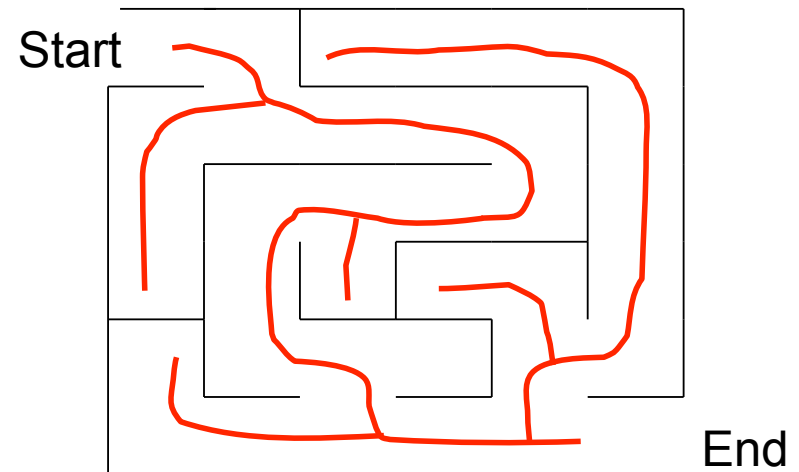
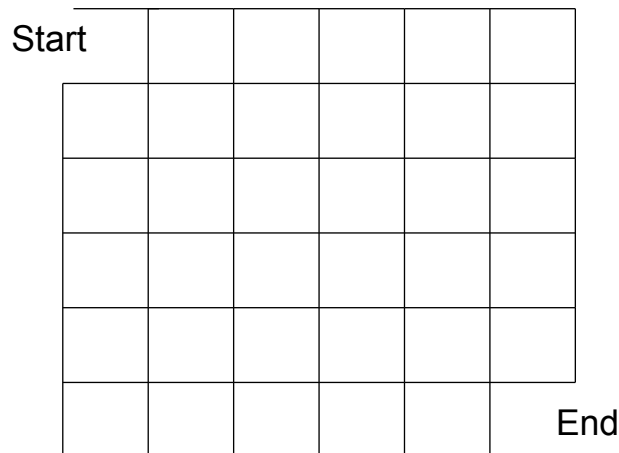
- Case 1: $\text{rank}(\text{parent}[x])$ is in higher bucket than $\text{rank}[x]$
 - At most $\log^*(n)$ links have this property
- Case 2: charge 1 credit to follow parent pointer
 - If $\text{rank}(x)$ is in bucket $\{B, 2^B - 1\}$, x will move to Case 1 before paying all its credits (and will still have some left over)
- So, since you started with $\leq n \log^*(n)$ credits across all nodes, and you spend less than that across all find operations, total amortized cost of m operations, for $m \geq n$, is $O(m \log^*(n))$

Example Application: Connected Components



Application

- Maze Building: take out walls to create a maze
 - Consider edges in random order (i.e. pick an edge)
 - Only delete an edge if it introduces no cycles (how? TBD)
 - When done, we will have a way to get from any place to any other place (including from start to end points)



Application 2

- Maze Building: take out walls to create a maze
 - Number the cells; every edge allows a relation between cells
 - Keep track of disjoint sets: don't remove edge if it is between two nodes in same subset

Start	1	2	3	4	5	6
	7	8	9	10	11	12
	13	14	15	16	17	18
	19	20	21	22	23	24
	25	26	27	28	29	30
	31	32	33	34	35	36
						End

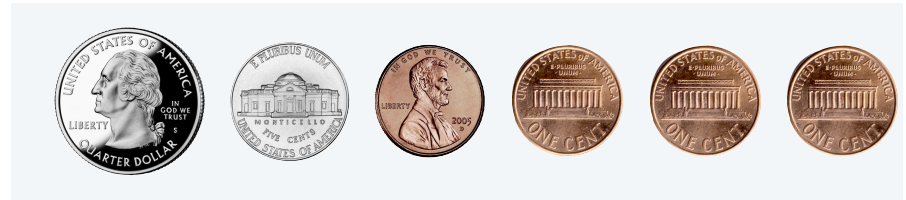
Start	1	2	3	4	5	6
	7	8	9	10	11	12
	13	14	15	16	17	18
	19	20	21	22	23	24
	25	26	27	28	29	30
	31	32	33	34	35	36
						End

Greedy Algorithms (Ch. 16, CLSR v3)

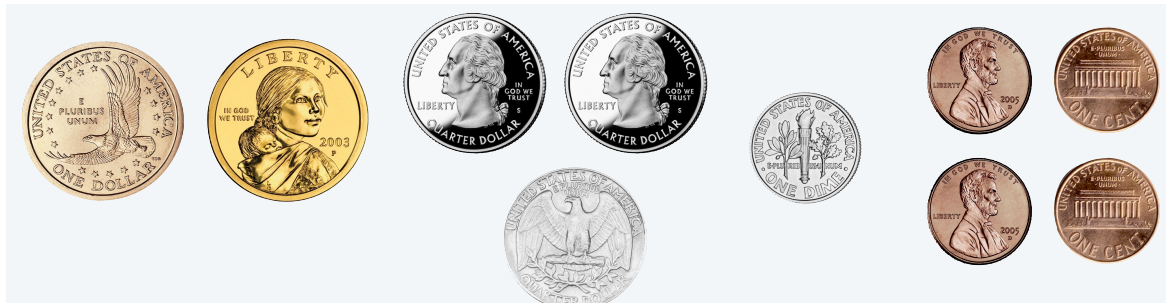
- Greedy algorithm: an algorithm that builds a solution adding an element at a time that is the locally optimal choice at that time
 - Uses a simple rule to select next element
 - No backtracking (!!!) so algorithms are fast (near-linear time)
 - Greedy algorithms are not always optimal for problems. When not optimal, may provide fast approximate algorithms...
 - Special classes of problems can be solved to optimality by greedy algorithms
 - Proving optimality is challenging

Example: Coin Changing

- Given U. S. currency denominations { 1, 5, 10, 25, 100 }, devise a method to pay amount using fewest coins



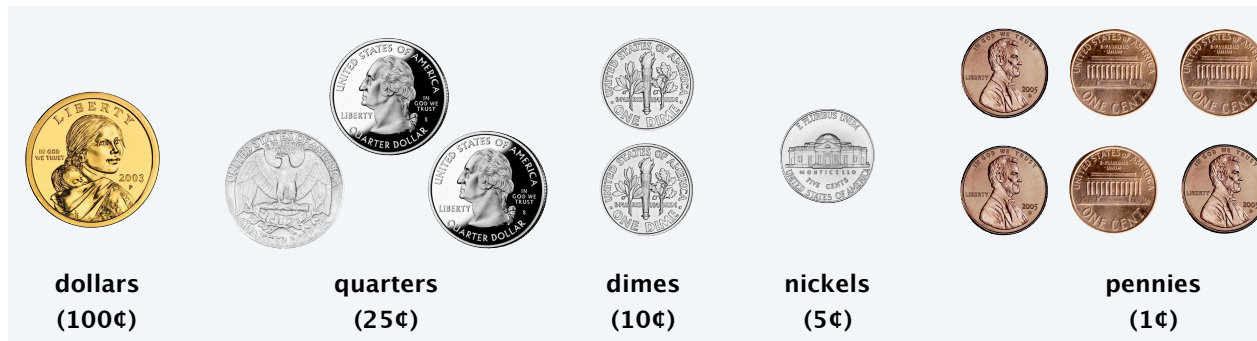
- Greedy Algorithm
 - At each iteration, add largest value coin that does not exceed amount
 - e.g. for \$2.89,



- Is it optimal? Will this yield fewest coins for any amount?

Greedy Coin Changing in US

- For US denominations, optimal solution must satisfy:
 - Number of pennies ≤ 4 (if not, can reduce coins with a nickel)
 - Number of nickels ≤ 1 (otherwise, use dime)
 - Number of quarters ≤ 3 (otherwise, use Liberty \$)
 - Number of nickels + dimes ≤ 2 (otherwise, use quarters)



Greedy Coin Changing Optimal in US

- Prove by induction: Let c_k = value of coin k , x = amount to be paid
 - Assume greedy is optimal for $x < c_k$
 - If $c_k \leq x < c_{k+1}$, optimal solution must include at least one of coin k

Otherwise, must match c_k with 1 lesser value coins, and it is not possible with US coins)

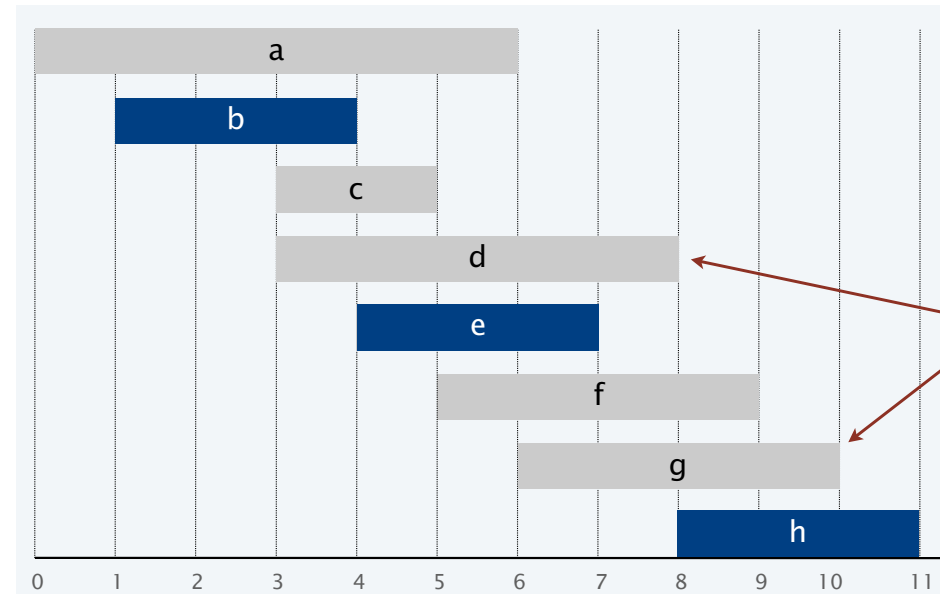
- Thus, optimal to use 1 of coin k , and then solve for $x - c_k$

k	c_k	all optimal solutions must satisfy	max value of coin denominations $c_1, c_2, \dots, c_{k-1}$ in any optimal solution
1	1	$P \leq 4$	–
2	5	$N \leq 1$	4
3	10	$N + D \leq 2$	$4 + 5 = 9$
4	25	$Q \leq 3$	$20 + 4 = 24$
5	100	<i>no limit</i>	$75 + 24 = 99$

- NOTE: not true for other denominations (e.g. stamps)

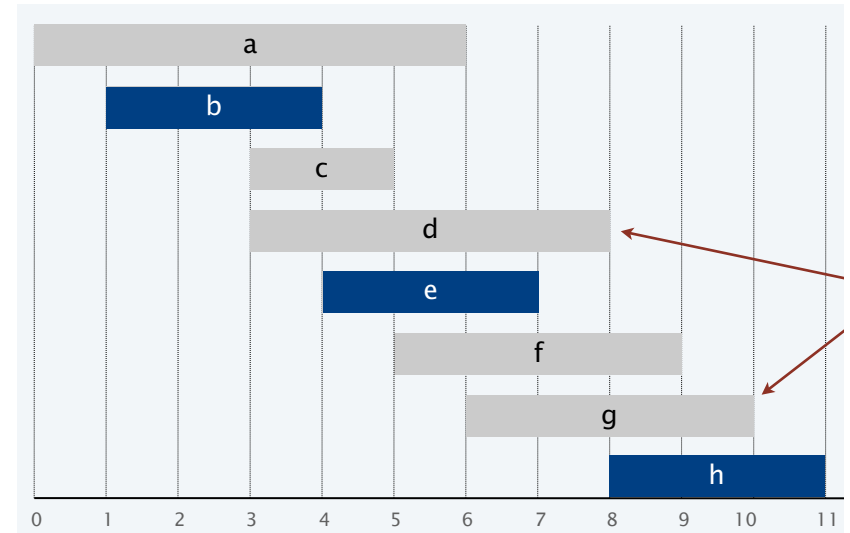
Interval Scheduling

- Jobs $\{1, 2, \dots, N\}$ occur at fixed times, with start/finish times $s(j)$, $f(j)$ for job j
- Single processor
- 2 jobs are compatible if they don't overlap
- Goal: Maximize number of jobs scheduled that are compatible



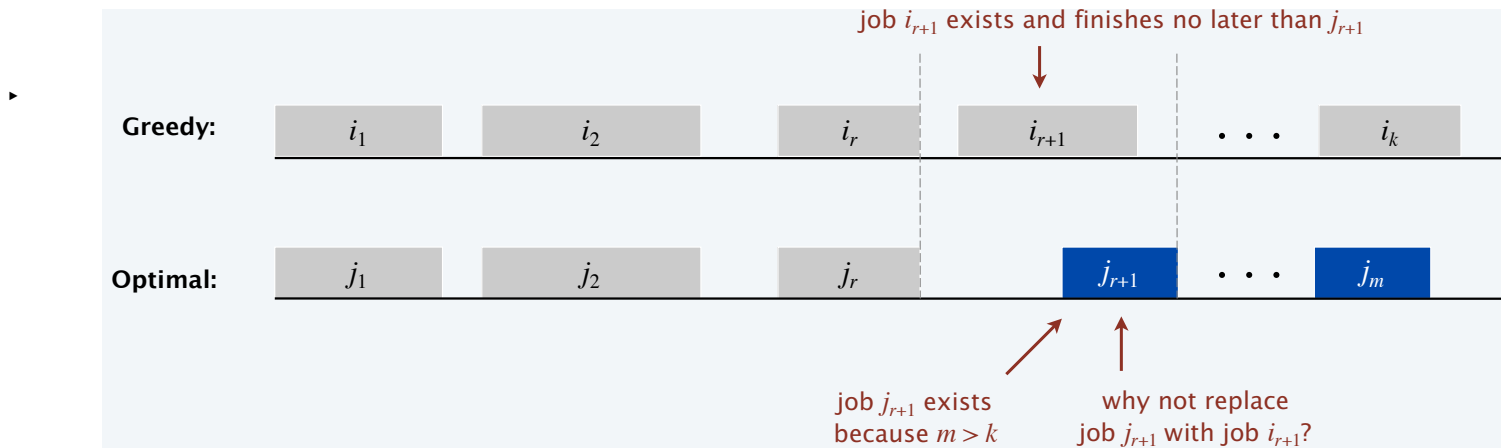
Interval Scheduling

- Quiz: Which is best? Sequence jobs by using an index, then add to schedule if compatible?
 - Earliest start time? (a,b,c,d,e,f,g,h)
 - Earliest finish time? (b,c,a,e,d,f,g,h)
 - Shortest interval? (c,b,e,h,f,g,d,a)
 - Least overlap? (h,b,c,g,a,e,f,d)



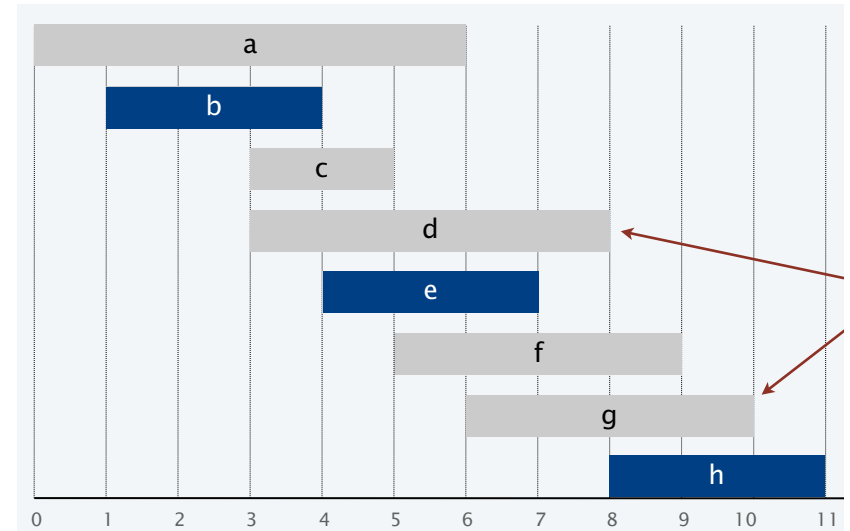
Interval Scheduling: Earliest finish time optimal

- Prove by contradiction
 - Let $i_1, i_2, \dots, i_k$ denote set of jobs selected by EFT, and let $j_1, j_2, \dots, j_m$ denote set of jobs in an optimal solution
 - Let r be largest value where $i_1 = j_1, \dots, i_r = j_r$ (schedules agree)
 - Note: i_{r+1} finishes no later than j_{r+1} (because of EFT choice)
 - So, can replace j_{r+1} with i_{r+1} and still be optimal for rest of schedule $J \rightarrow$ contradiction



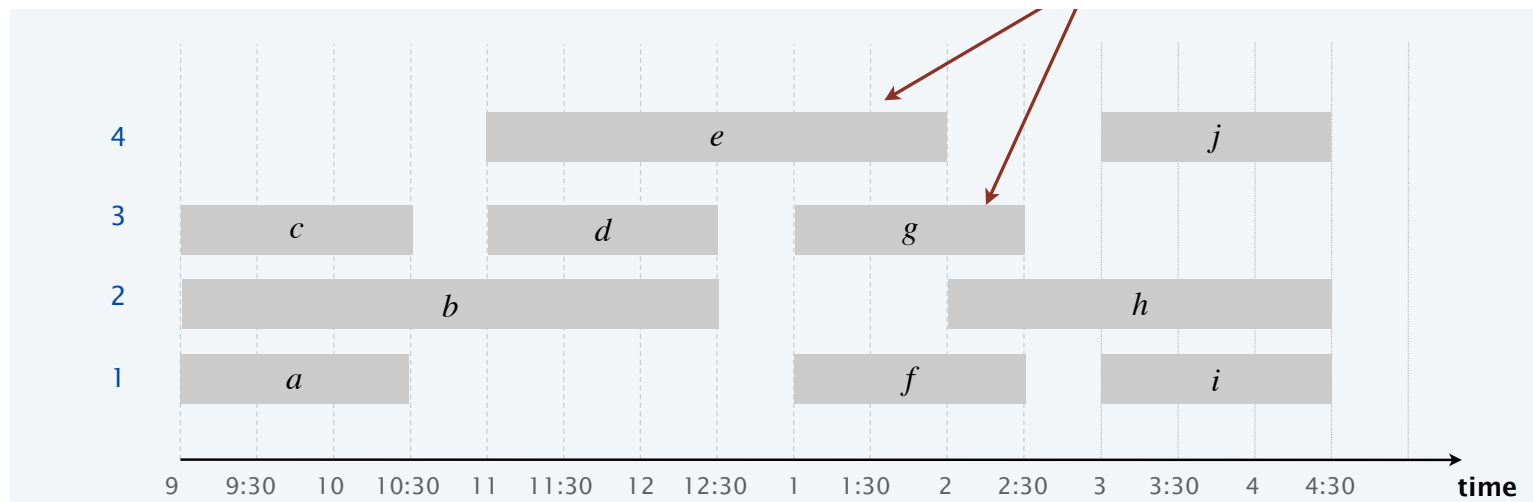
Parallel Job Scheduling

- Quiz: Which is best? Sequence jobs by using an index, then assign to first available processor?
 - Earliest start time? (a,b,c,d,e,f,g,h)
 - Earliest finish time? (b,c,a,e,d,f,g,h)
 - Shortest interval? (c,b,e,h,f,g,d,a)
 - Least overlap? (h,b,c,g,a,e,f,d)
 - ????



Scheduling Jobs on Parallel Processors

- Jobs $\{1, 2, \dots, N\}$ occur at fixed times, with start times $s(j)$, $f(j)$ for job j
- What is minimum number of parallel processors so we can schedule all jobs without conflict?
- The registrar problem: jobs are classes with start times, end times, and processors are classrooms (a bit more complicated if you add class sizes)



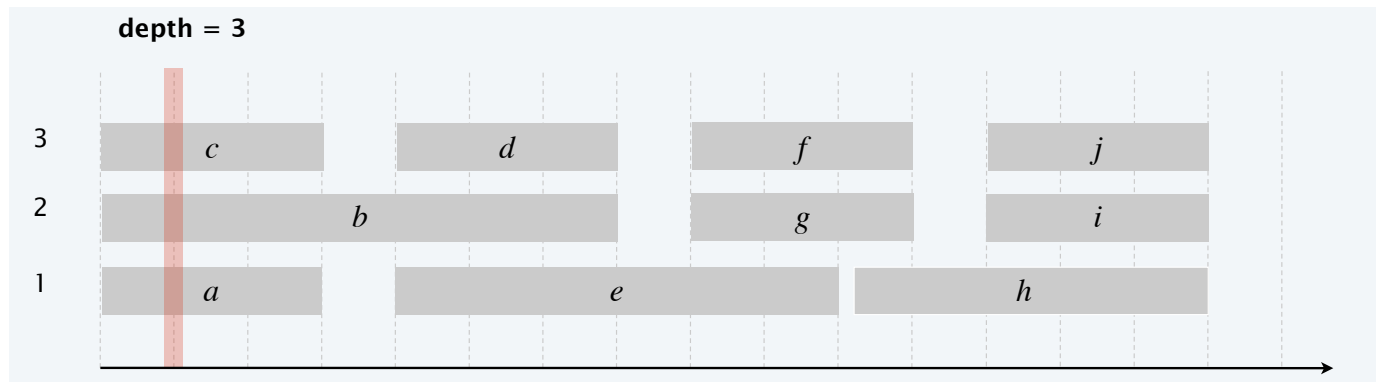
Optimal Algorithm

- Sequence by earliest start time: $s(1) \leq s(2) \leq \dots \leq s(n)$
 - Let d = number of allocated classrooms
 - For $j = 1$ to n :
 - If processor available at $s(j)$, assign $s(j)$ to processor
 - Else increase $d = d+1$, assign $s(j)$ to new processor
- Alternative, sequence by earliest finishing time...would also work
- $O(n \log(n))$ $\rightarrow$ need to sort

▸

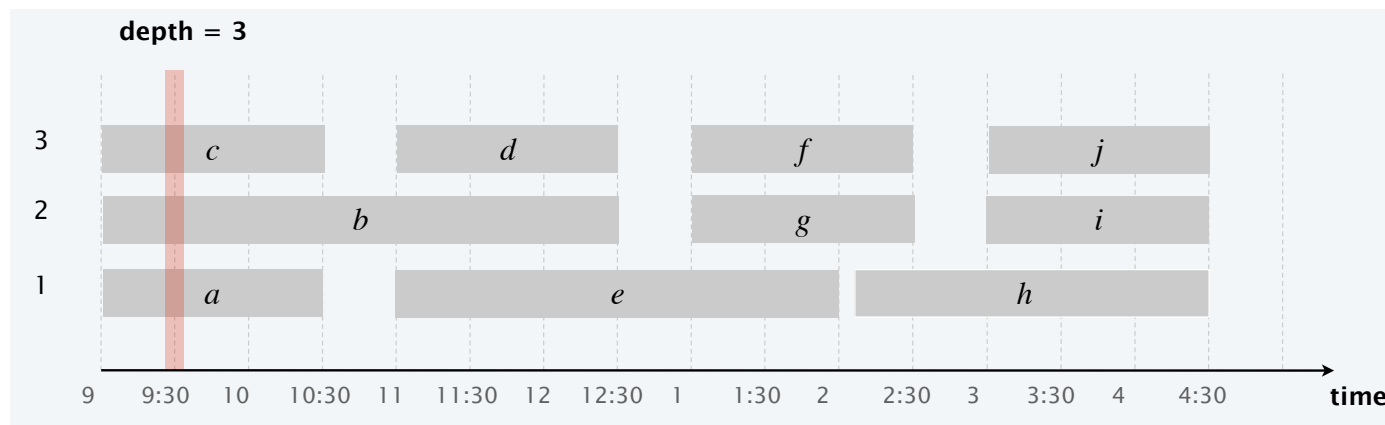
Concept: Depth of intervals

- Def. The depth of a set of open intervals is the maximum number of intervals that contain any given point.
- Key observation. Number of processors needed $\geq$ depth.
- Q. Does minimum number of processors needed always equal depth?
 - Yes! Moreover, earliest-start-time-first algorithm finds a schedule whose number of processors equals the depth



Optimality

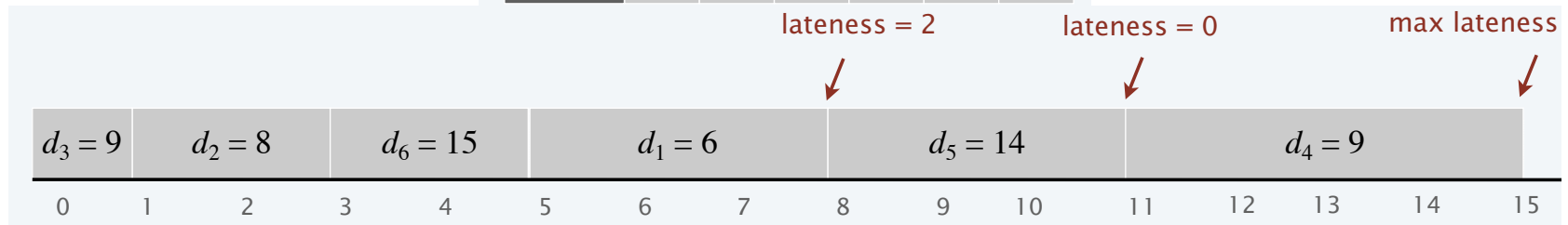
- If you must add a processor to increase d in earliest-start-time algorithm for job j at $s(j)$, then depth is at least $d+1$ at time $s(j) + \epsilon$
- Key observation. Number of processors needed $\geq$ depth
- Thus, EST algorithm achieves lower bound on number of processors, hence it is at least as good as any other algorithm $\rightarrow$ it is optimal



Scheduling to minimize lateness

- Jobs $\{1, 2, \dots, N\}$, with required processing times t_j and deadlines d_j
- If job j starts at time s , it finishes at $f_j = s + t_j$
- Lateness measure $\ell_j = \max\{0, f_j - d_j\}$
- Goal: schedule all jobs to minimize maximum lateness $L = \max_{j=1, \dots, N} \ell_j$

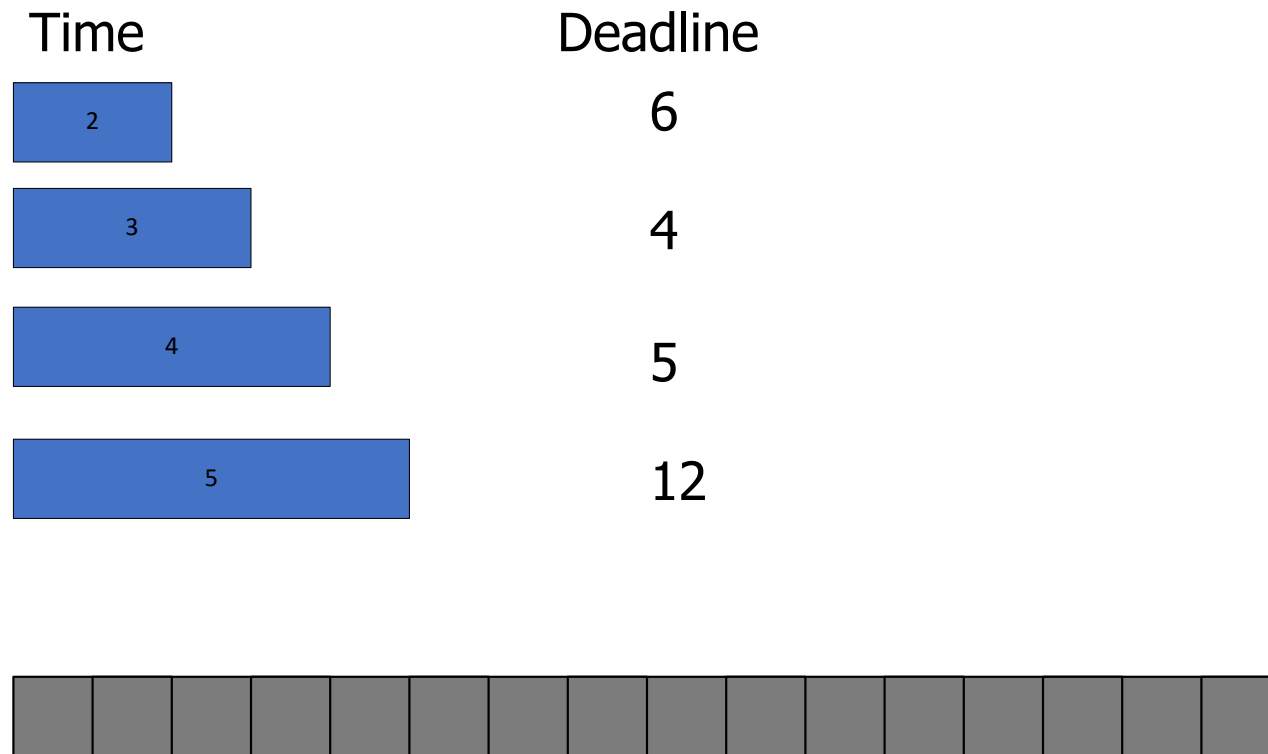
	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



Example

Time	Deadline
	2
	4
	Lateness 1
	Lateness 3

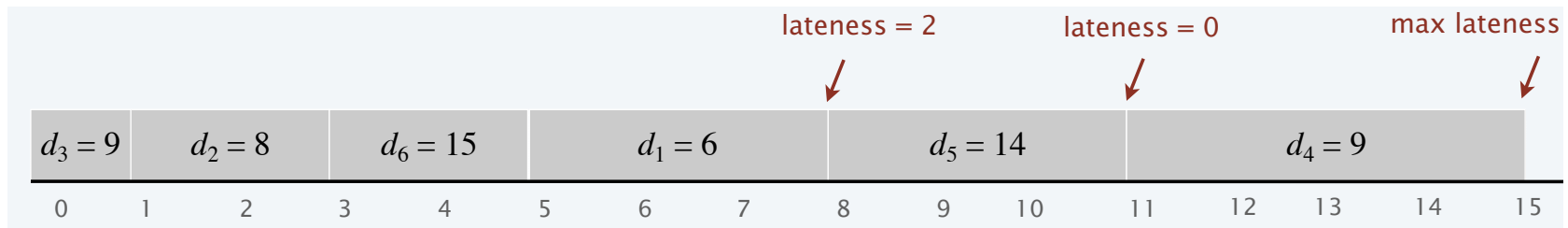
Determine the minimum lateness



Deadline Scheduling

- Quiz: Which is best? Sequence jobs by using an index?
 - Shortest processing time?
 - Earliest deadline first?
 - Increasing order of slack $d_j - t_j$?
 - Something else?

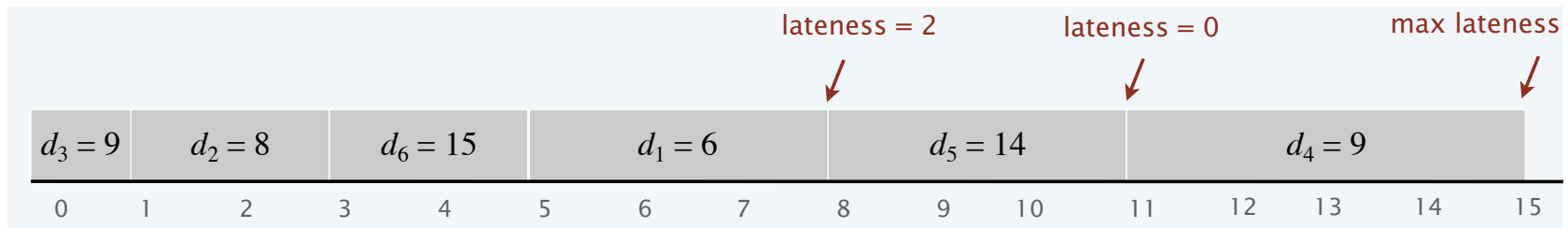
	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



Greedy algorithm: Earliest deadline first

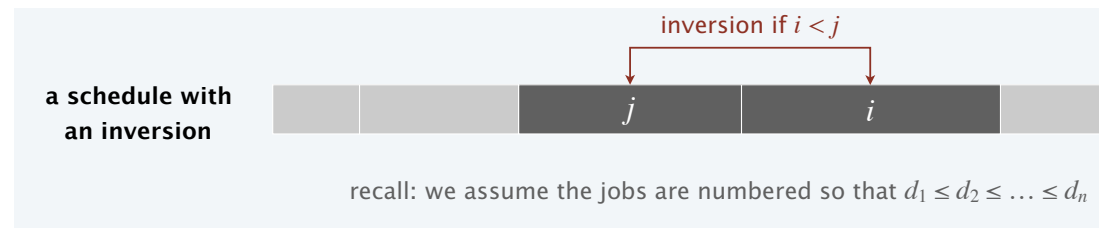
- Sort jobs by deadline, so $d_1 \leq d_2 \leq \dots \leq d_N$
- Start with $t = 0$
- For $j=1$ to N :
 - Schedule job j in $[t, t+d_j]$
 - Increase $t = t+d_j$

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



Observation: No idle time is optimal

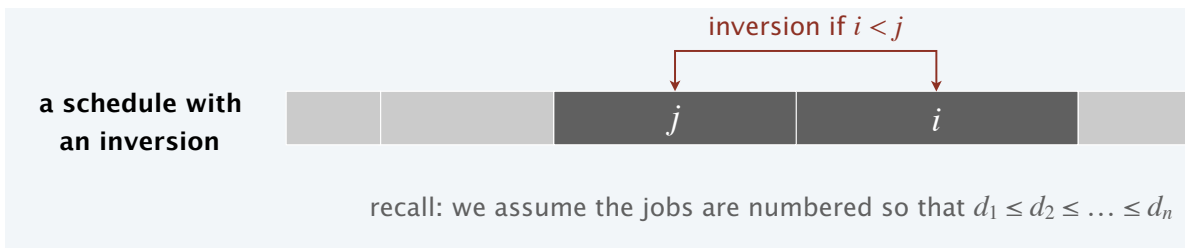
- There exist an optimal schedule with no idle time (by contradiction, easy to perturb to eliminate idle time)
 - Earliest deadline schedule has no idle time
- Definition: a schedule has an inversion if, for a particular pair $j < k$, job k is scheduled before job j
 - Note: if an inversion exists, then there must exist a pair of adjacent jobs in a schedule with inversion



- Earliest deadline schedule has no inversions

Minimizing Lateness

- Given a schedule S with inversions, exchanging the order of an adjacent, inverted pair j, k does not increase maximum lateness
- Proof: Let ℓ_m be the lateness of job m before the exchange, and ℓ'_m the lateness after the exchange
 - $\ell'_m = \ell_m$ for all $m \neq j, k$
 - $\ell'_j \leq \ell_j$ because we have moved it up
 - If job k is now late, $\ell'_k = f'_k - d_k = f_j - d_k \leq f_j - d_j = \ell_j$ (earliest deadline)
 - So, max lateness is not increased



Optimality

- Earliest deadline first is optimal
- Proof:
 - Assume there is an optimal schedule that is not earliest deadline first that has the fewest number of inversions among optimal schedules
 - If number of inversions is 0, it is the EDF solution.
 - If not, there must be an adjacent inversion
 - Swapping the inversion does not increase the maximum lateness, and reduces the number of inversions, so we generate an optimal schedule which has less inversions
 - Contradiction

Minimize Total Finishing Time

- Single server, N jobs
 - Job j has processing times $t(j)$
 - Select start time $s(j)$ for j, when server is idle; finish time $f(j) = s(j) + t(j)$
 - Objective: Schedule jobs to minimize total finishing time summed over jobs

$$\text{Min } \sum_{j=1}^N f(j)$$

- Optimal solution: Greedy algorithm
 - Sort jobs by increasing processing time so $t(1) \leq t(2) \leq \dots \leq t(N)$ and schedule back-to-back in that order

Proof of Optimality

- Proof by contradiction:

Let P is an optimal schedule. Let p_j be job in position j in P

Assume P is not ordered according to increasing processing time

Find first j such that $t_{p_j} > t_{p_{j+1}}$ (must exist because of P ordering)

Swap these in schedule to get schedule P' , and show P' is better

Note: finishing time of $p_1, \dots, p_{j-1}$ and $p_{j+1}, \dots, p_N$ is the same under P, P'

Let T_{j-1} be finishing time p_{j-1} . Then,

Under P , finishing times of p_j and p_{j+1} are: $T_{j-1} + t_{p_j}$, $T_{j-1} + t_{p_j} + t_{p_{j+1}}$

Under P' , finishing times of p_j and p_{j+1} are: $T_{j-1} + t_{p_{j+1}}$, $T_{j-1} + t_{p_j} + t_{p_{j+1}}$

Since $t_{p_j} > t_{p_{j+1}}$, P' is better! So P cannot be optimal, contradiction.

Example

- Jobs: 1, 2, 3, 4, 5;
- Processing times: 5, 4, 3, 2, 3
- Algorithm: Sort jobs by processing time: 4, 3, 5, 2, 1, schedule in order

4	4	3	3	3	5	5	5	2	2	2	2	1	1	1	1	1	
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--

- Finishing times: $2 + 5 + 8 + 12 + 17 = 44$
- Alternative schedule: schedule by number: 1, 2, 3, 4, 5

1	1	1	1	1	2	2	2	2	3	3	3	4	4	5	5	5	
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--

- Finishing times: $5 + 9 + 12 + 14 + 17 = 57$

Scheduling with Deadlines

- Single server, N jobs with the same processing time
 - Job j has **unit processing time**, hard deadline $d(j)$, value $V(j)$ if job completed on time
 - Objective: Find sequence of jobs that will be scheduled to maximize value completed on time
 - Concept: a set of jobs $\{j_1, \dots, j_k\}$ is feasible $\iff$ there is an ordering of these jobs where each job is completed before its deadline
 - If a set of jobs is feasible, scheduling them by earliest deadline first is sufficient $\rightarrow$ simple to show
 - Can also schedule them just in time — schedule job j as close as possible to its deadline $d(j)$

Determining Best Feasible Set

- Algorithm:
 - Initialize feasible set S as empty; sort the jobs by decreasing value $V(j)$, into order $j_1, \dots, j_N$
 - Initialize step $k = 1$
 - At step k , add job j_k to S , and see if S is still a feasible set
 - If S is not feasible, remove j_k from S
 - If $k < N$, increment $k = k+1$ and repeat
- Claim: this will be an optimal feasible set
 - Important feature: every job takes the same amount of time

Proof of optimality

- Let S : feasible set constructed by algorithm. T : optimal feasible set that has the most overlap with S
- Assume T has more value than S , and assume the jobs in T are indexed according to decreasing value
- Assume that all jobs that are **both** in S and T are scheduled at the same times
 - This can always be done, by scheduling jobs as late as possible
- Find the highest value job in S that is not in T , denoted j^* . Swap that job with whatever job is scheduled at the same time in T . You get a no worse T^* , so T^* is as optimal as T or more so
 - Why is T^* feasible? Because j^* is scheduled before its deadline in S , hence also in T^*
 - Why is T^* better? Because S selected feasible sets by decreasing value, so value of j^* at least as large as value of any job in T that is not in S
 - Why do we get a contradiction? Because T^* is at least optimal, and has more overlap with S than T ; but T had the highest overlap with S among optimal sequences

Example

Jobs	1	2	3	4	5	6	7	8
Values	5	4	3	2	3	4	2	1
Deadlines	3	5	2	4	3	6	5	7

- Algorithm: Sort by value: 1, 2, 6, 3, 5, 4, 7, 8
- Find feasible set by greedy algorithm:
 - Scheduling just-in-time (as late as possible):



7

String Matching and Tries

- Consider the following problem:
 - Given a string T and k nonempty strings $P_1, P_2, \dots, P_k$, find all occurrences of $P_1, P_2, \dots, P_k$ in T .
 - T is called the text string and $P_1, P_2, \dots, P_k$ are called pattern strings.
 - This problem was originally studied in the context of compiling indexes, but has found applications in computer security and computational genomics
- Simpler problem: $k = 1$
 - Knuth-Morris-Pratt (KMP) algorithm (CLRS)
- Harder problem: $k > 1$
 - Aho-Corasick tries

Single Pattern Matching

- Given a string $T[0:n-1]$ of n characters in finite alphabet set S
- Nonempty string $P = P[0:m-1]$ of characters in S
- Pattern P occurs with shift s in $T[s:s+m-1] = P[0:m-1]$
- Problem: Find all shifts s where Pattern occurs with shift s
- Naive algorithm:
 - For s in 0 to $n-m$:
 - If $T[s:s+m-1] = P[0:m-1]$, add s to set of shifts where pattern occurs
 - Complexity: $O(m(n-m))$

Single Pattern Matching

- Example:
 - $T[] = \text{"AAAAAAAAAAAAAAAAAAB"} \quad (n=18)$
 - $P[] = \text{"AAAAB"} \quad (m=5)$

One shift matches: $s=13$

- Example
 - $T[] = \text{"ABABABCABABABCABABABC"} \quad (n=20)$
 - $P[] = \text{"ABABAC"} \quad (m=6)$

No shifts match!

Knuth-Morris-Pratt Algorithm

- Idea: exploit that whenever we find a mismatch, we have already looked at a subset of the pattern
 - $T[] = \text{"AAAA A AAAAAAAAAAAAB"} \quad (n=20)$
 - $P[] = \text{"AAAA B"} \quad (m=6)$
 - If we get a mismatch on $T[5]$, we don't have to start searching from $s=1$ for the next match! We know no shift can match before $s = 6$
- Idea: patterns have a prefix function
 - A string w is a **proper prefix** of a string x if $x = w + v$ for some string non-empty string v , where $+$ is concatenation
 - A string w is a **proper suffix** of a string x if $x = v + w$ for some non-empty string v

Knuth-Morris-Pratt Algorithm

- Algorithm: preprocess pattern $P[0:m-1]$ to compute prefix function $\pi[j]$
 - $\pi[j]$ = longest proper prefix of $P[0:j]$ that is also a suffix of $P[0:j]$, for $j = 0$ to $m-1$
 - Examples: $P = \text{"AAAA"} \rightarrow \pi[] = [0, 1, 2, 3]$
 - $P = \text{"ABCDE"} \rightarrow \pi[] = [0, 0, 0, 0, 0]$
 - $P = \text{"AABAACAABAA"} \rightarrow \pi[] = [0, 1, 0, 1, 2, 0, 1, 2, 3, 4, 5]$
 - $P = \text{"AAACAAAAAC"} \rightarrow \pi[] = ?$
 - $P = \text{"AAABAAA"} \rightarrow \pi[] = ?$
- We can use the prefix function to figure out how much to shift the pattern forward for matching!

Knuth-Morris-Pratt Algorithm

- Analysis

- String P: "a b a b a b a b c a"; Prefix function: $\pi(i)$: [0,0,1,2,3,4,5,6,0,1]
- Match into T: "a b a b **d** a a b a b a b a b c a d c d"

```
a b a b d a a b a b a b a b c a d c d
a b a b a b a b c a
```

Shift **to check P at** $\pi(3) = 2$, try again

```
a b a b d a a b a b a b a b c a d c d
      a b a b a b a b c a
```

Shift **to check P at** $\pi(1) = 0$, try again

```
a b a b d a a b a b a b a b c a d c d
      a b a b a b a b c a
```

No match, move on

```
a b a b d a a b a b a b a b c a d c d
              a b a b a b a b c a
```

Shift **to check P at** $\pi(1) = 0$, try again

```
a b a b d a a b a b a b a b c a d c d
              a b a b a b a b c a
```

Found a match at start = 6

Knuth-Morris-Pratt Algorithm

- Computing $\pi[j]$ from $P[0:m-1]$:

$k = 0; j = 1;$

$\pi[0] = 0;$

While $j < m$:

 If $P[k] == P[j]$:

$k++; \pi[j] = k; j++;$

 else if $k == 0$: # character did not match

$\pi[j] = 0; j++;$

 else: #mismatch after character

$k = \pi[k-1]$

w

c	a	b	c	a
---	---	---	---	---

arr

0	0	0	1	2
---	---	---	---	---

P	A	A	B	A	A	C	A	A	B
Pi	0	1	0	1	2	0	1	2	3
K	0	1	0	1	2	0	1	2	3
J	1	2	3	4	5	6	7	8	9

Knuth-Morris-Pratt Algorithm

```

k = 0; j = 0;
while (j < n)
    if P[k] == T[j]
        k++; j++;
    if k == m:
        print("Found pattern at index ", j-k)
        k=pi[k-1]
    else if (j < n and P[k] != T[j]):
        if k != 0
            k= pi[k-1]
        else
            j++;
return
    
```

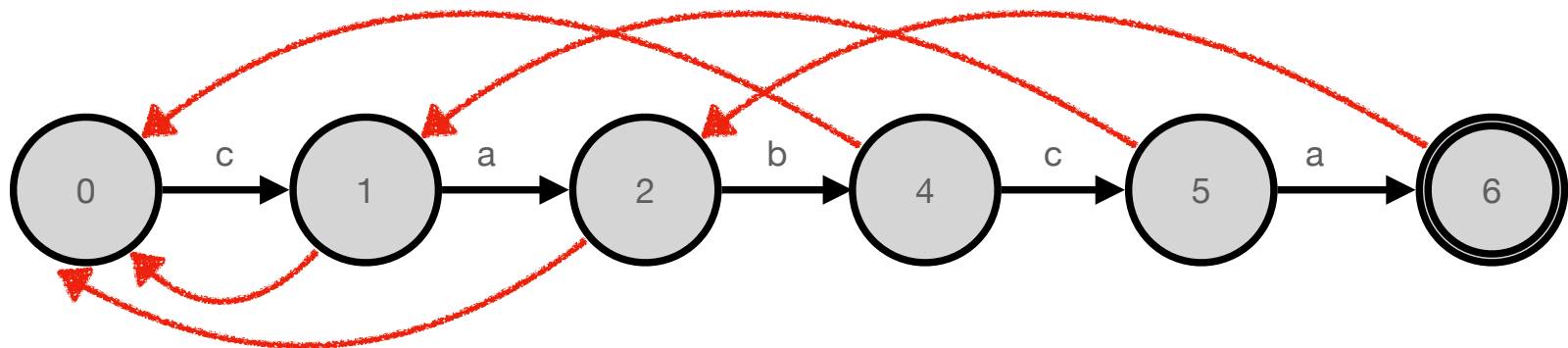
T	A	A	B	A	A	A	A	A	B
P	A	A							
pi	0	1							
s	1	0	0	1	1	1	1	0	0
k	0/1	1/0	0	0/1	1/0	1/0	1/0	1/0	0
j	0	1	2	3	4	5	6	7	8

T	A	A	B	A	A	A	A	A	B
P	A	A	A						
pi	0	1	2						
s	0	0	0	1	1	1	0	0	0
k	0/1	1/2	2/0	0/1	1/2	2/1	2/1	2/1	0
j	0	1	2	3	4	5	6	7	8

A Different View of the Prefix Function

Finite State Automata

Pattern: c a b c a $\pi() = [0,0,0,1,2]$



- If letter match, advance to next state, check next character.
- Else, follow red link to state, check current character.
- If reach double circle, output match, follow red link to previous state and continue searching current character.

Knuth-Morris-Pratt Algorithm

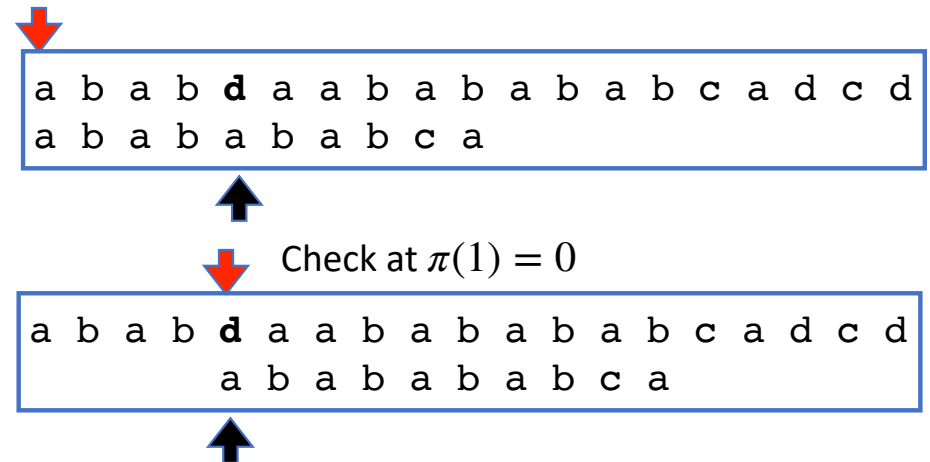
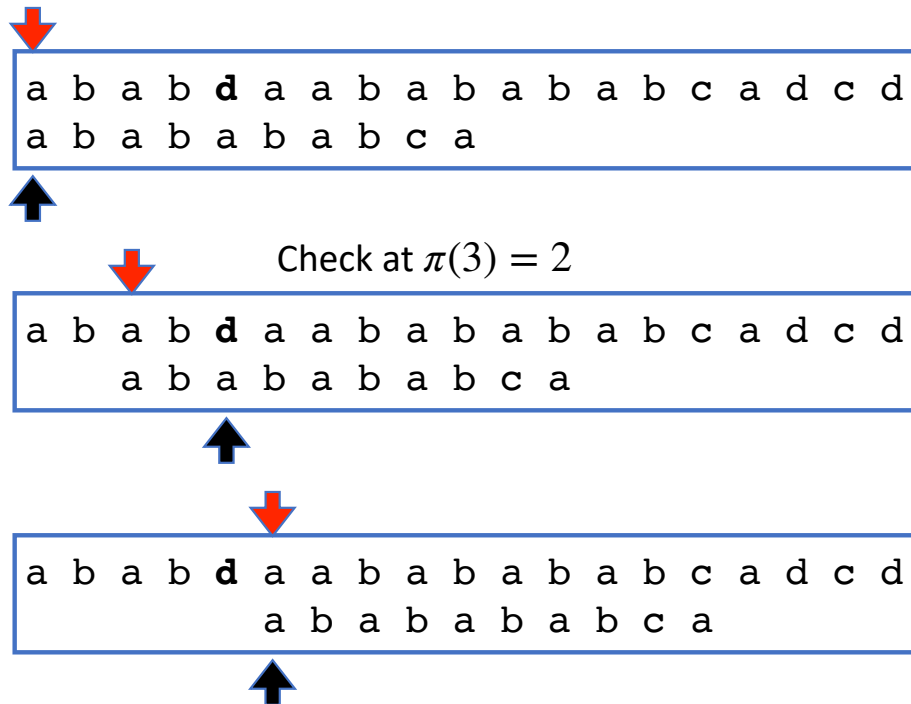
- Complexity analysis
 - For each position j in the string T :
 - If it does not match, you slide the pattern to all possible suffixes and check for a match
 - Worst case pattern: "AAAAAAA": $\pi[k] = k$ so lots of prefixes
- Let q = number of digits matched when checking position j .
 - Potential $\Phi(j) = q$. Note $\Phi(0) = 0$, $\Phi(end) = 0$
 - Cost of checking position j : $c(j)$ = number of iterations of $k = \pi[k - 1]$ for this j
 - Cost is $1 + c(j)$
 - Each iteration $k = \pi[k - 1]$ decreases number matched by at least one
 - Amortized cost for position j : $1 + c(j) + \Phi(j) - \Phi(j - 1) = 1 + c(j) - c(j)$: $\Theta(1)$
- To do n iterations: $\Theta(n)$

Knuth-Morris-Pratt Algorithm

- Complexity analysis with arrows

- String P: "a b a b a b a b c a"; Prefix function: $\pi(i)$: [0,0,1,2,3,4,5,6,0,1]

- Match into T: "a b a b **d** a a b a b a b a b c a d c d"



For every check, one of the two arrows moves at least one step forward: Total algorithm is $O(n)$

Knuth-Morris-Pratt Algorithm

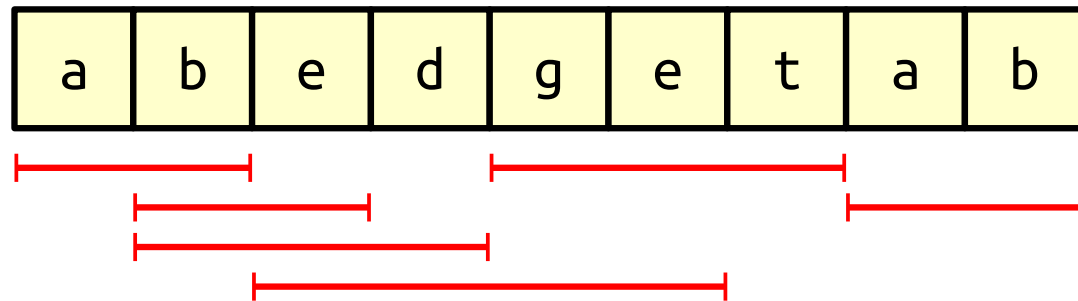
- Demo: <https://cmpps-people.ok.ubc.ca/ylucet/DS/KnuthMorrisPratt.html>

Search for Multiple Strings

- KMP good for 1 pattern P
- What if we are looking for k patterns $P_1, P_2, \dots, P_k$?

Pattern Strings

a	b			
a	b	o	u	t
a	t			
a	t	e		
b	e			
b	e	d		
e	d	g	e	
g	e	t		

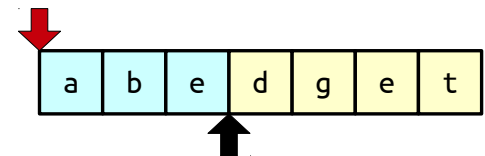
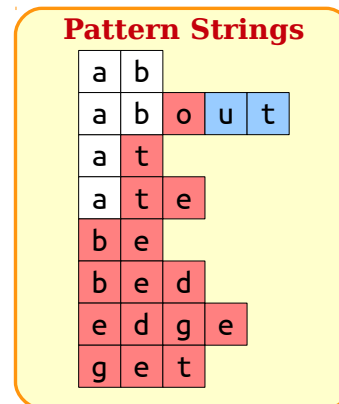
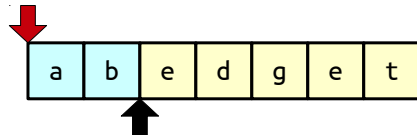
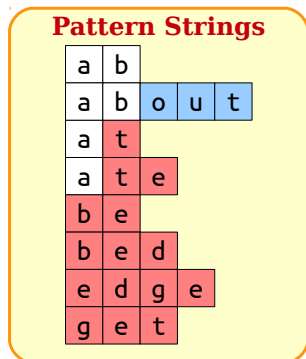
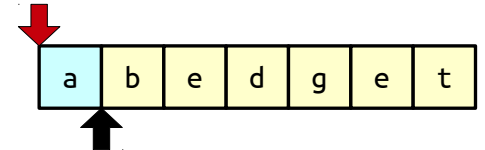
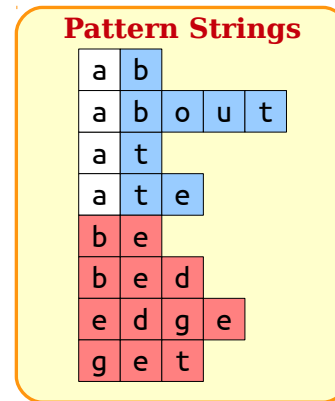
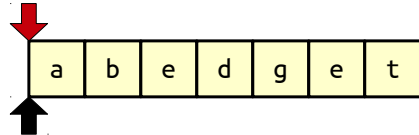
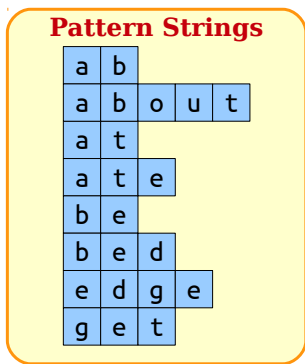


Search for Multiple Strings

- Notation: m is T string length; n is total length of k patterns; L_{max} length of longest pattern
- Naive approach:
 - For each position in T :
 - For each pattern string P_j :
 - Check if P_j appears at that position
 - $\Theta(mn)$
 - Can we do better?

Search for Multiple Strings

- Can we do better?

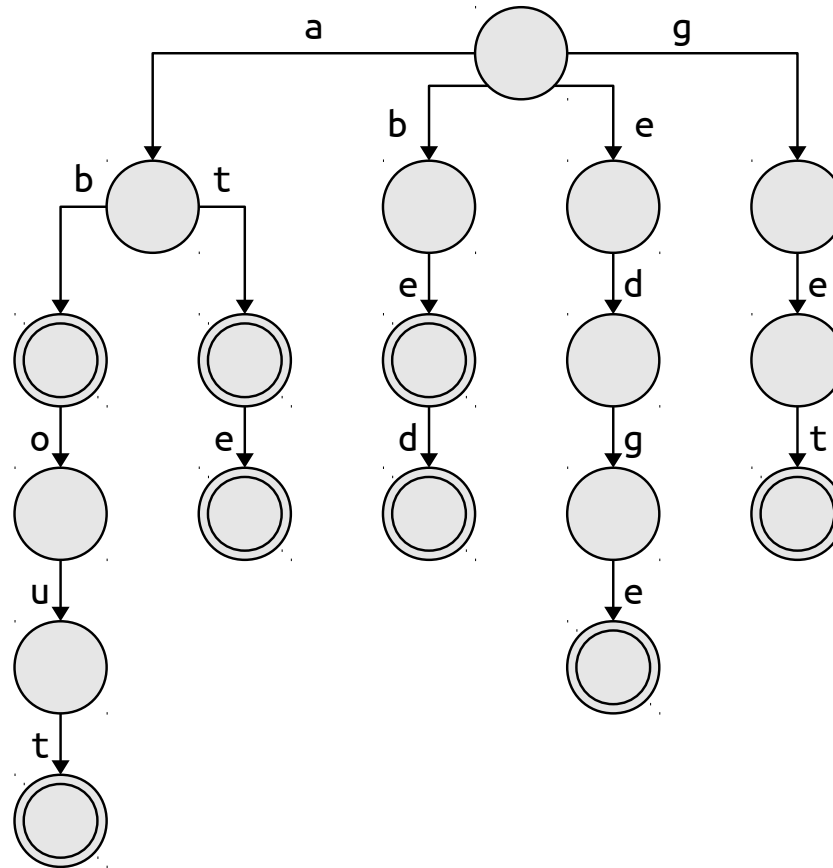
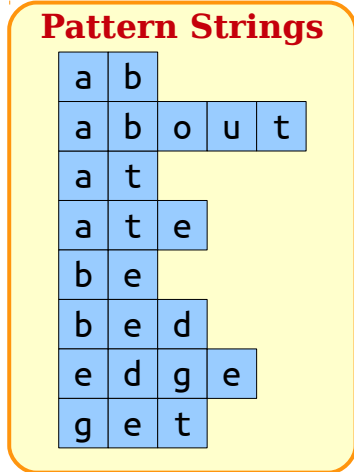


Search for Multiple Strings

- Idea: Search for all strings in parallel
- How? Form the strings into a **trie**: word comes from **retrieval**
- A trie: a tree structure where the branches at each level correspond to symbol values
 - Example trie: index tables at end of documents
 - Important idea: In a trie, you wait to subdivide a node until there is a collision of two elements...saves storage! Can identify nodes with key words as potential end nodes

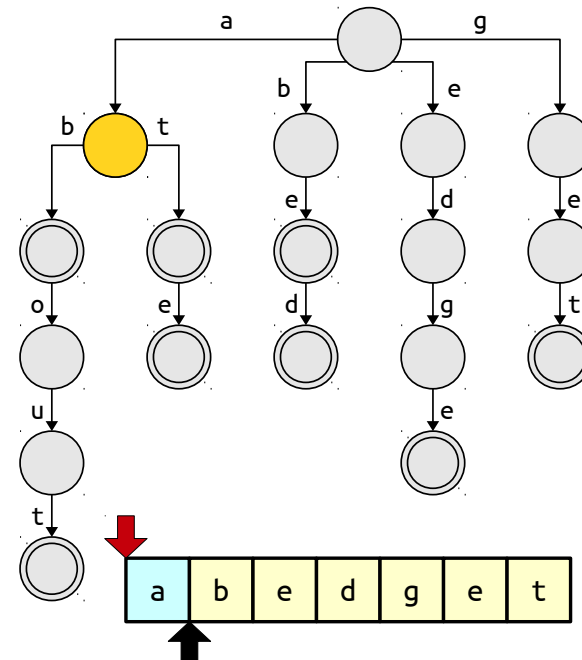
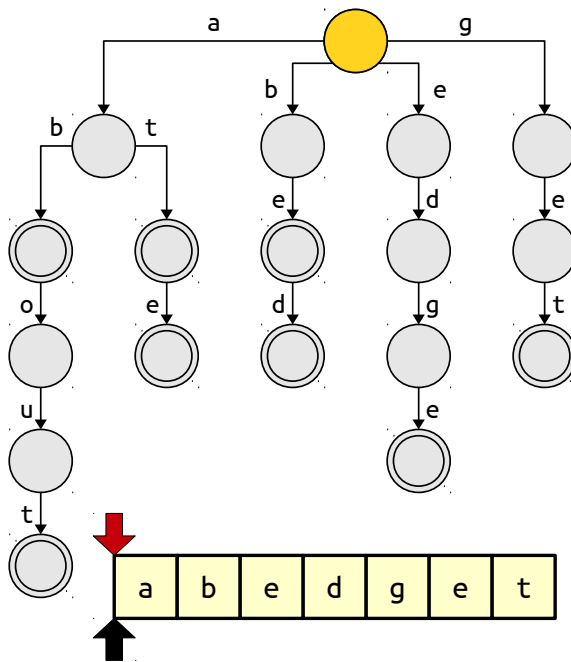
Search for Multiple Strings

- Example trie for words

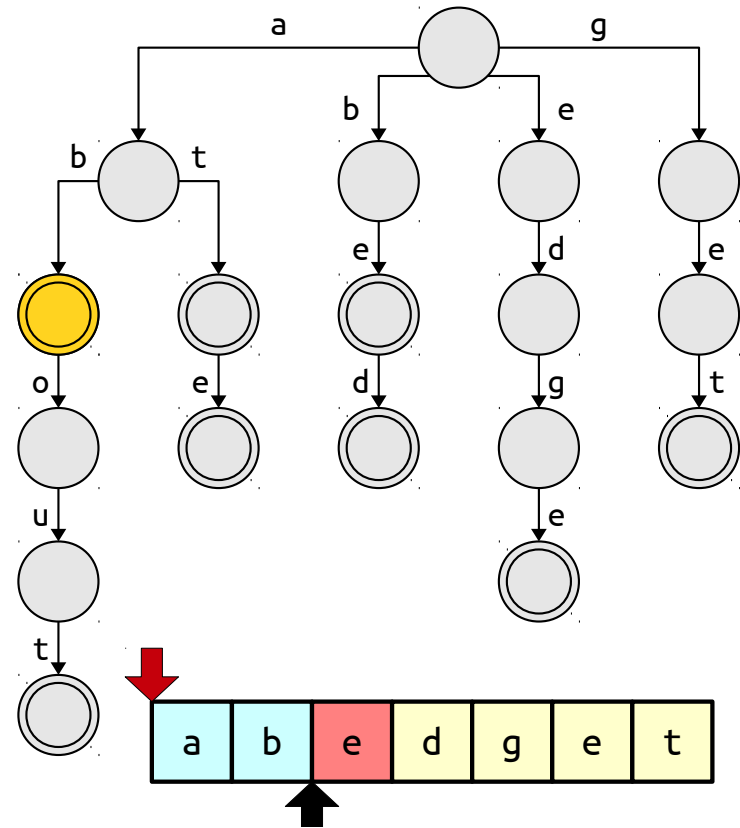
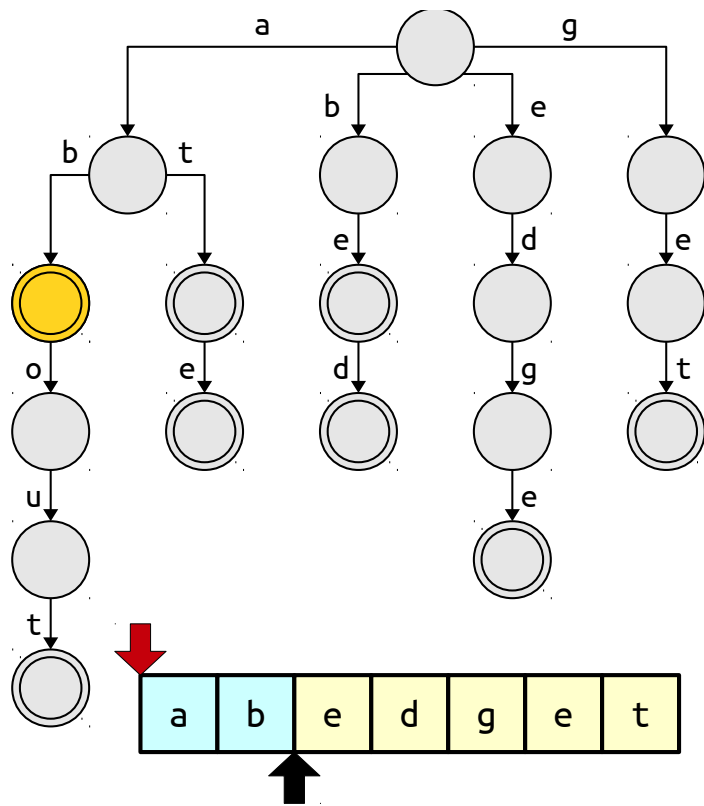


Representing Tries

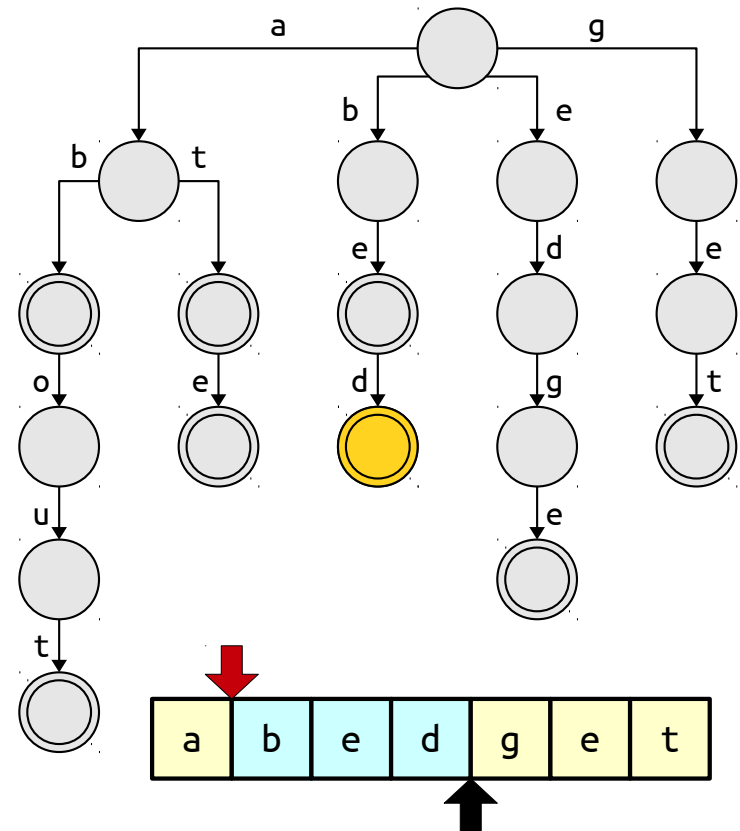
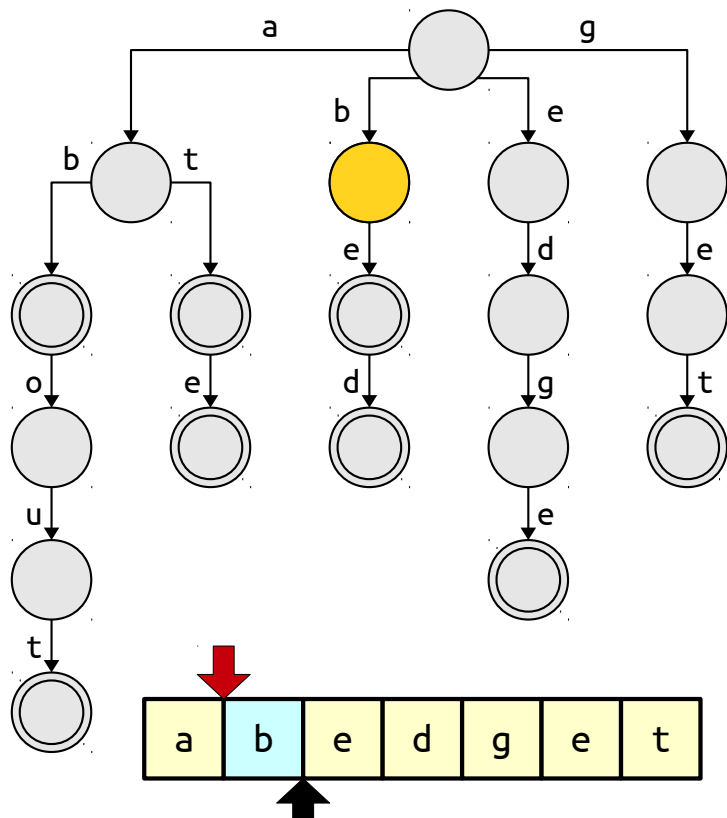
- Each trie node has pointers to its possible children
 - Assume you have an array of pointers at each node, many of which could be null
- Matching:



Matching Tries

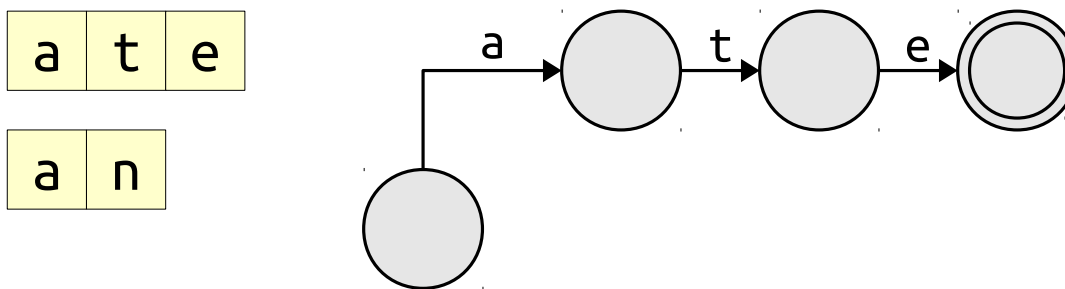


Matching Tries



Analysis of algorithm

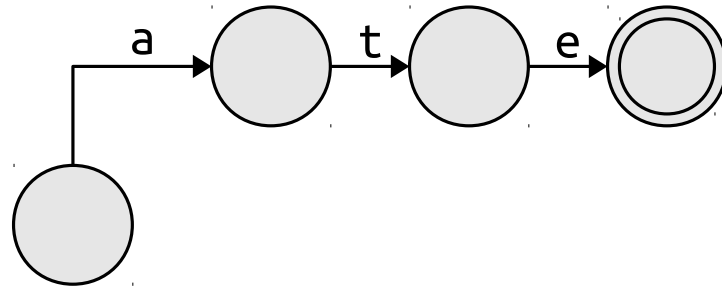
- Complexity: $\Theta(mL_{max})$
 - Good reduction if n is large (several patterns)
- But: How long to build a trie? Need that preprocessing step
- Claim: Given a set of strings $P_1, P_2, \dots, P_k$ of total length n , it's possible to build a trie for those strings in time $\Theta(n)$



Building Tries

a t e

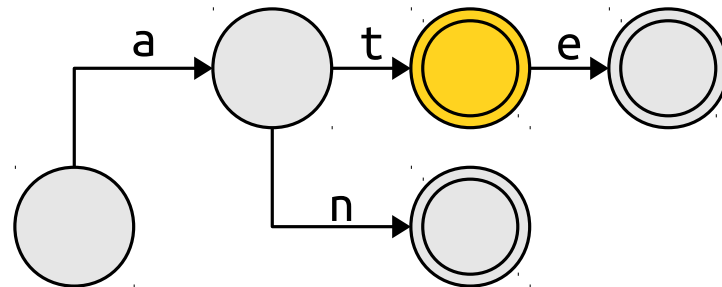
a n



a t e

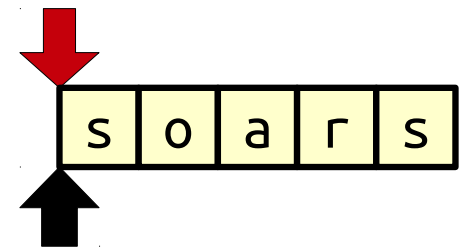
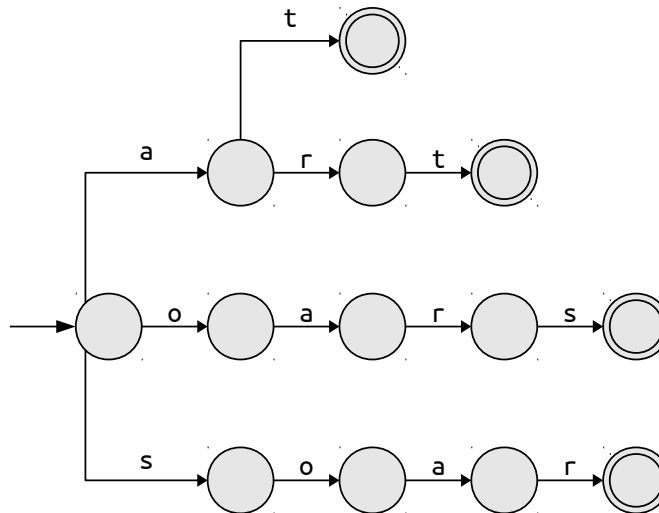
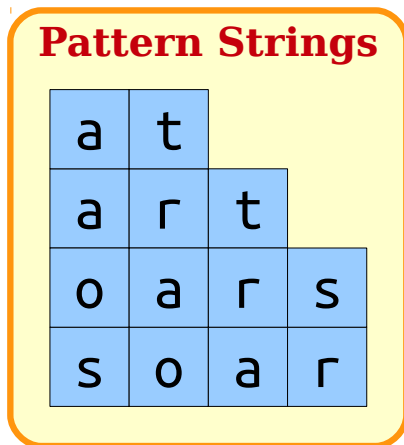
a n

a t

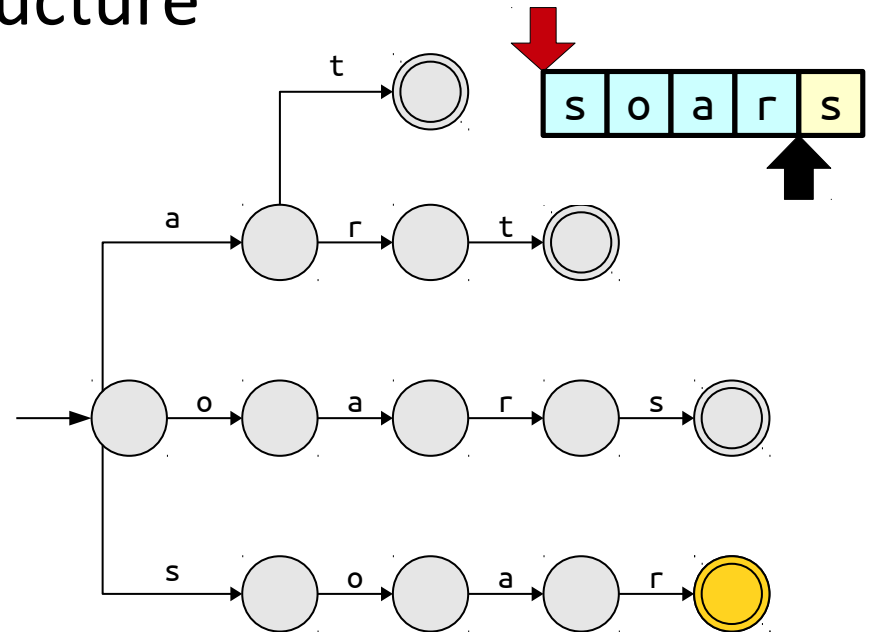
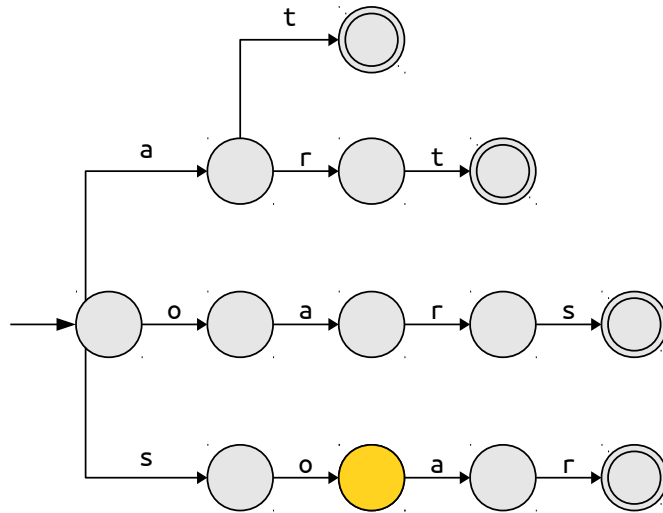
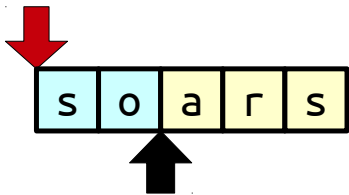
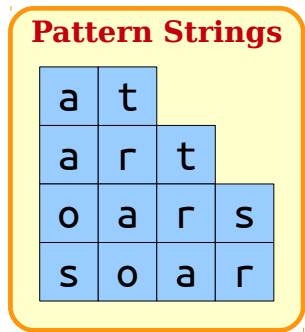


Analysis of algorithm

- Two algorithms so far
 - Naive algorithm: no preprocessing run time $\Theta(mn)$
 - Trie algorithm: $\Theta(n)$ preprocessing, $\Theta(mL_{max})$ run time
- Can we do better? Can we exploit prefix functions, as in KMP?



Exploiting structure

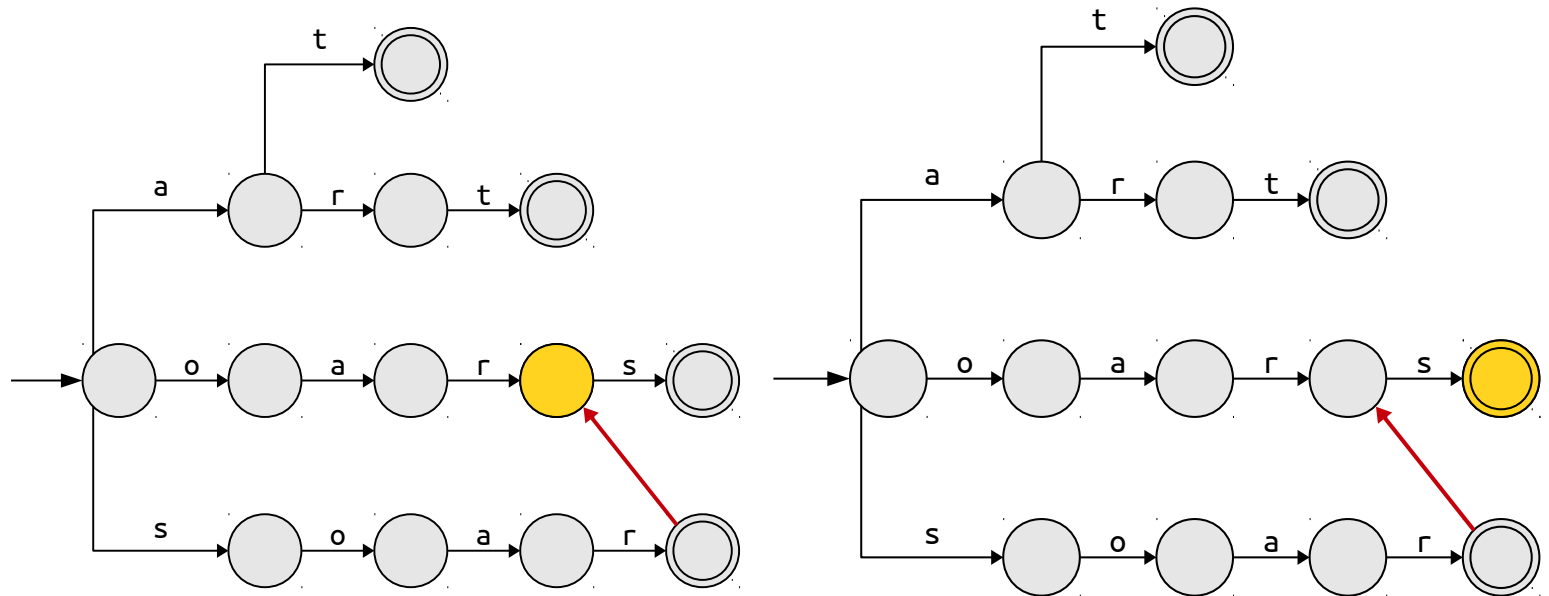
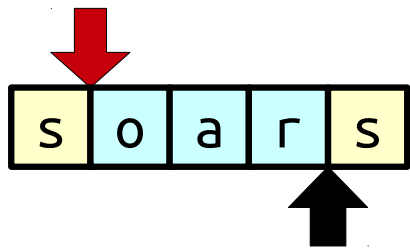


Observe: Suffix oar is prefix in oars
 Want to exploit that!

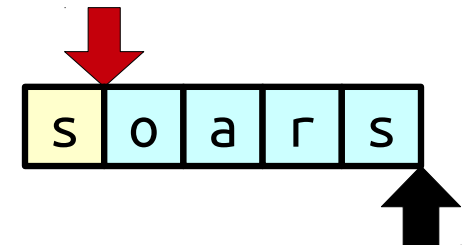
Exploiting structure

Pattern Strings

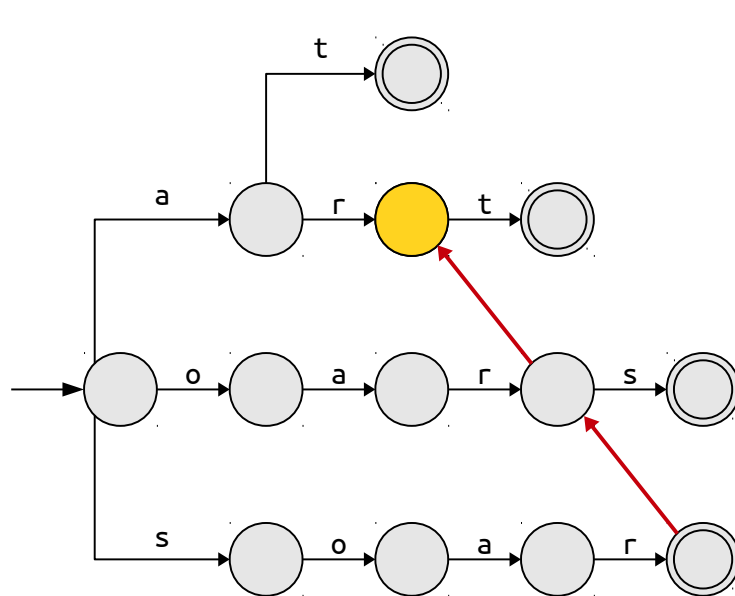
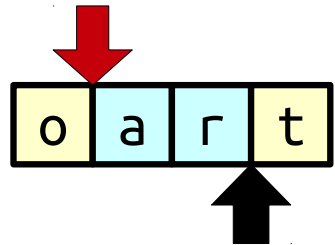
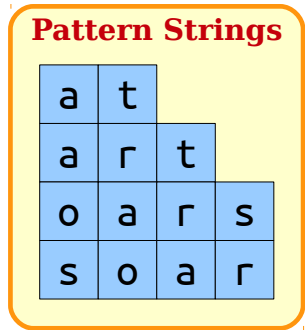
a	t		
a	r	t	
o	a	r	s
s	o	a	r



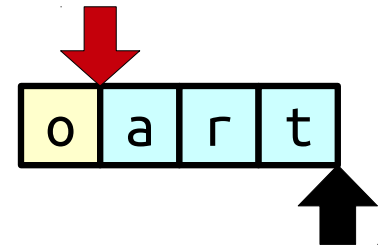
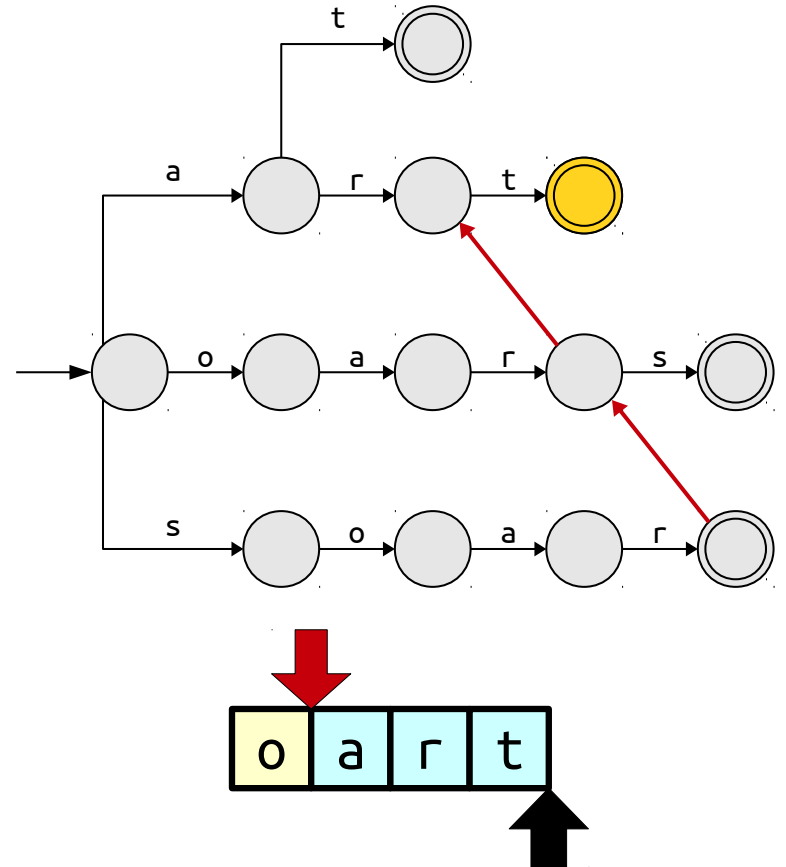
Red link is a suffix link



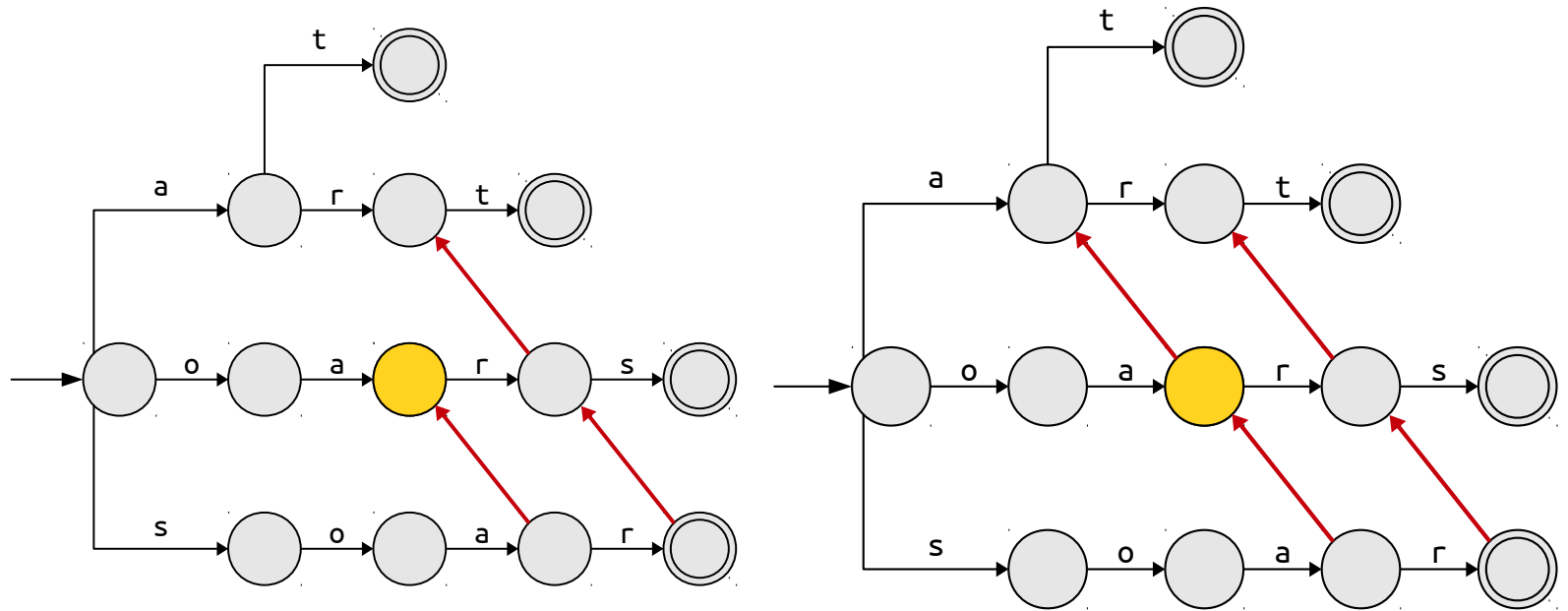
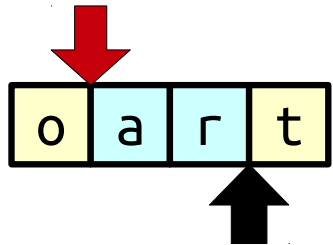
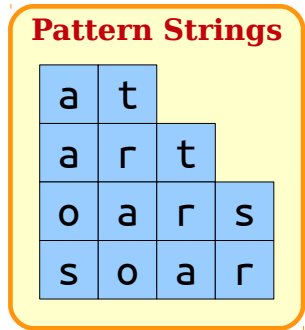
Exploiting structure



Red link is a suffix link

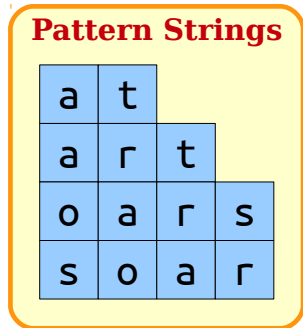


Exploiting structure

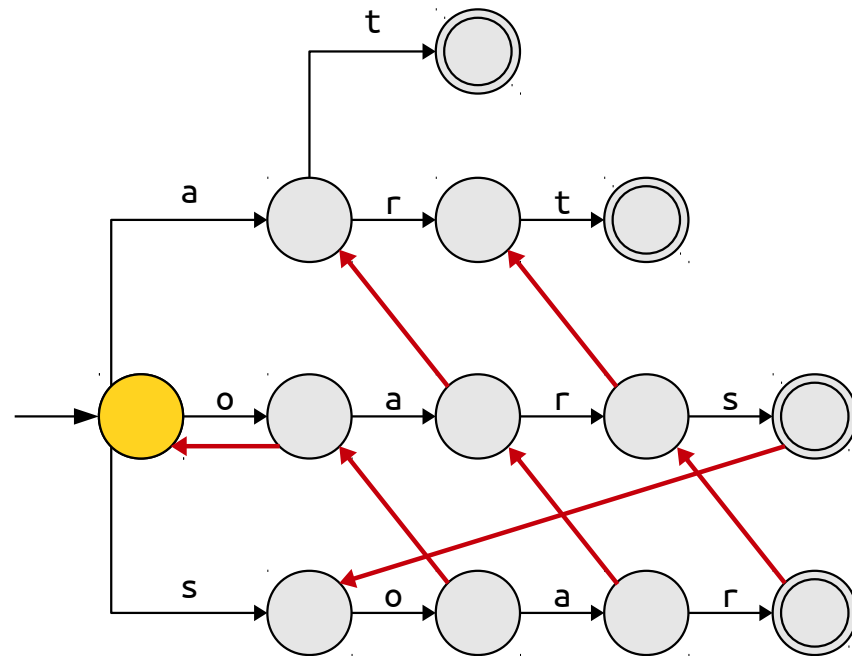


Red link is a suffix link

Exploiting structure



s o t a t



Red link is a suffix link

Suffix Links

- A **suffix link** is a red edge from a trie node corresponding to string a to the trie node corresponding to a string ω such that ω is the longest proper suffix of a that is still in the trie
- Intuition: When we hit a part of the string where we cannot continue to read characters, we fall back by following suffix links to try to preserve as much context as possible
- Every node in the trie, except the root (which corresponds to the empty string ε), will have a suffix link associated with it
 - We don't display suffix links back to the root, assumed implicit if there is no suffix link out of internal node.

Suffix Links Are Useful

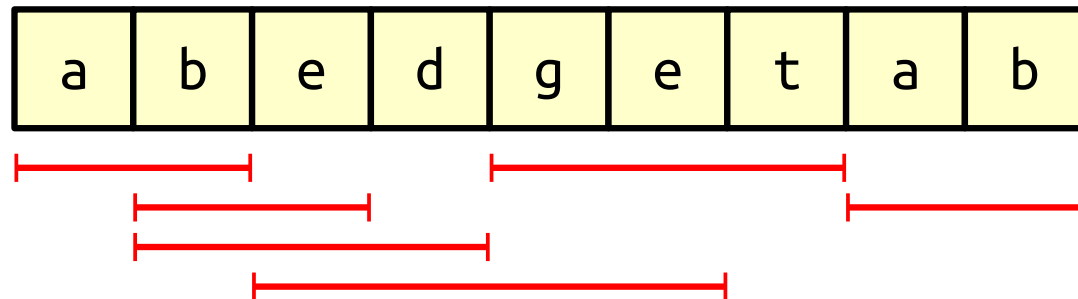
- Suffix links can substantially improve the performance of our string search
 - At each step, we either advance the black (end) pointer forward in the trie, or advance the red (start) pointer forward
- Each pointer can advance forward at most $O(m)$ times
- This reduces the amount of time spent scanning characters from $O(mL_{max})$ down to $\Theta(m)$
- This is only useful if we can compute suffix links quickly... which we'll see how to do it

Search for Multiple Strings

- KMP good for 1 pattern P
- What if we are looking for k patterns $P_1, P_2, \dots, P_k$?

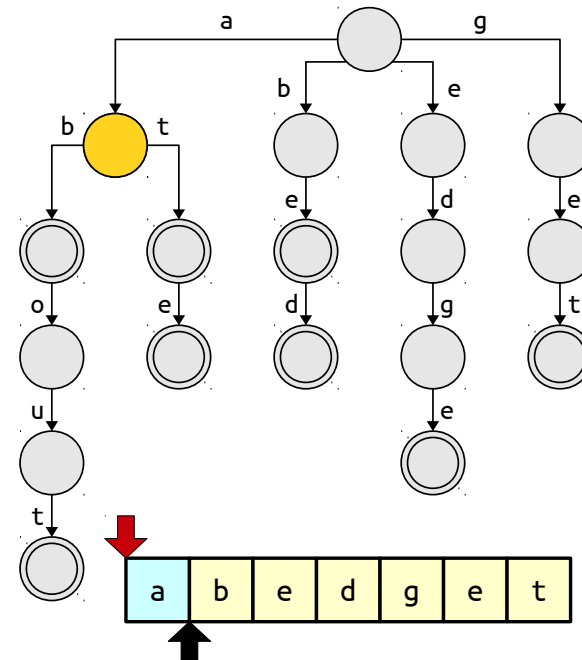
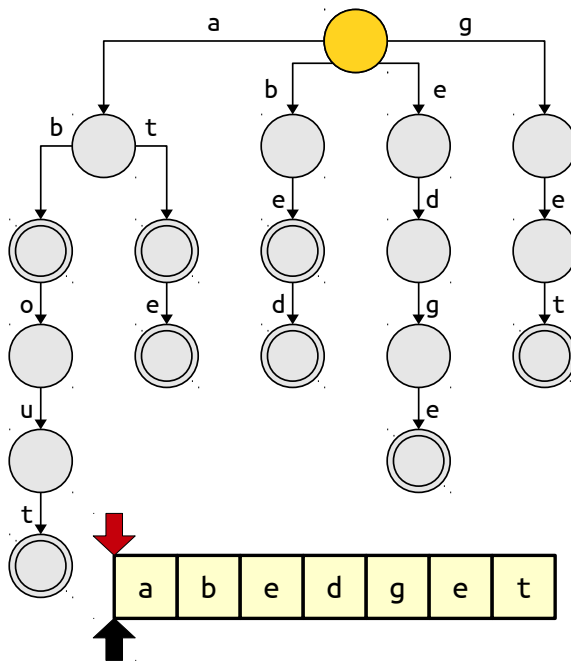
Pattern Strings

a	b			
a	b	o	u	t
a	t			
a	t	e		
b	e			
b	e	d		
e	d	g	e	
g	e	t		

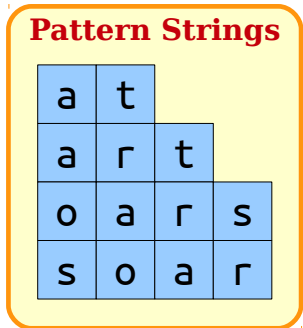


Idea: Construct a trie for all the search words

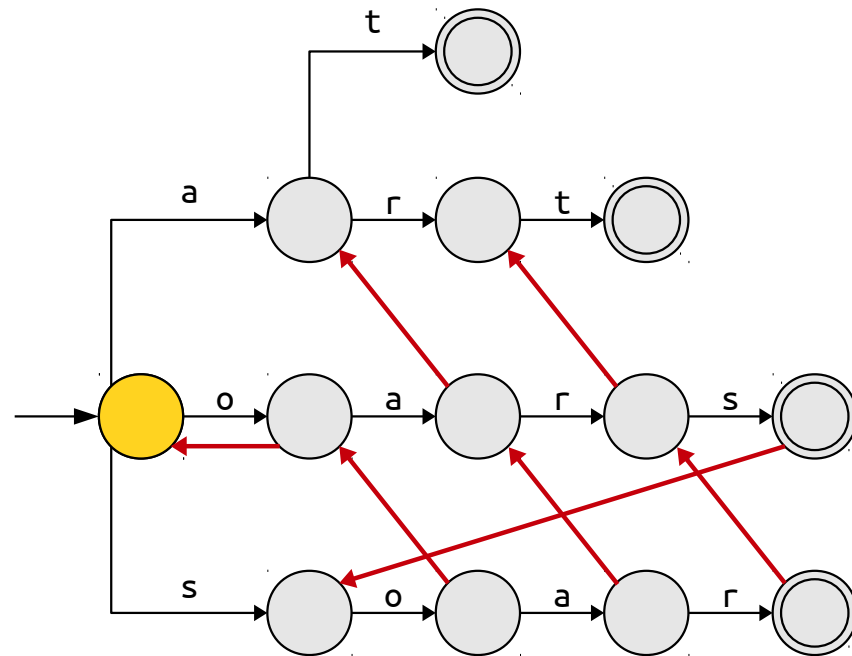
- Each trie node has pointers to its possible children
 - Assume you have an array of pointers at each node, many of which could be null
- Matching:



Using the trie, develop suffix links to reuse prefix of words already detected

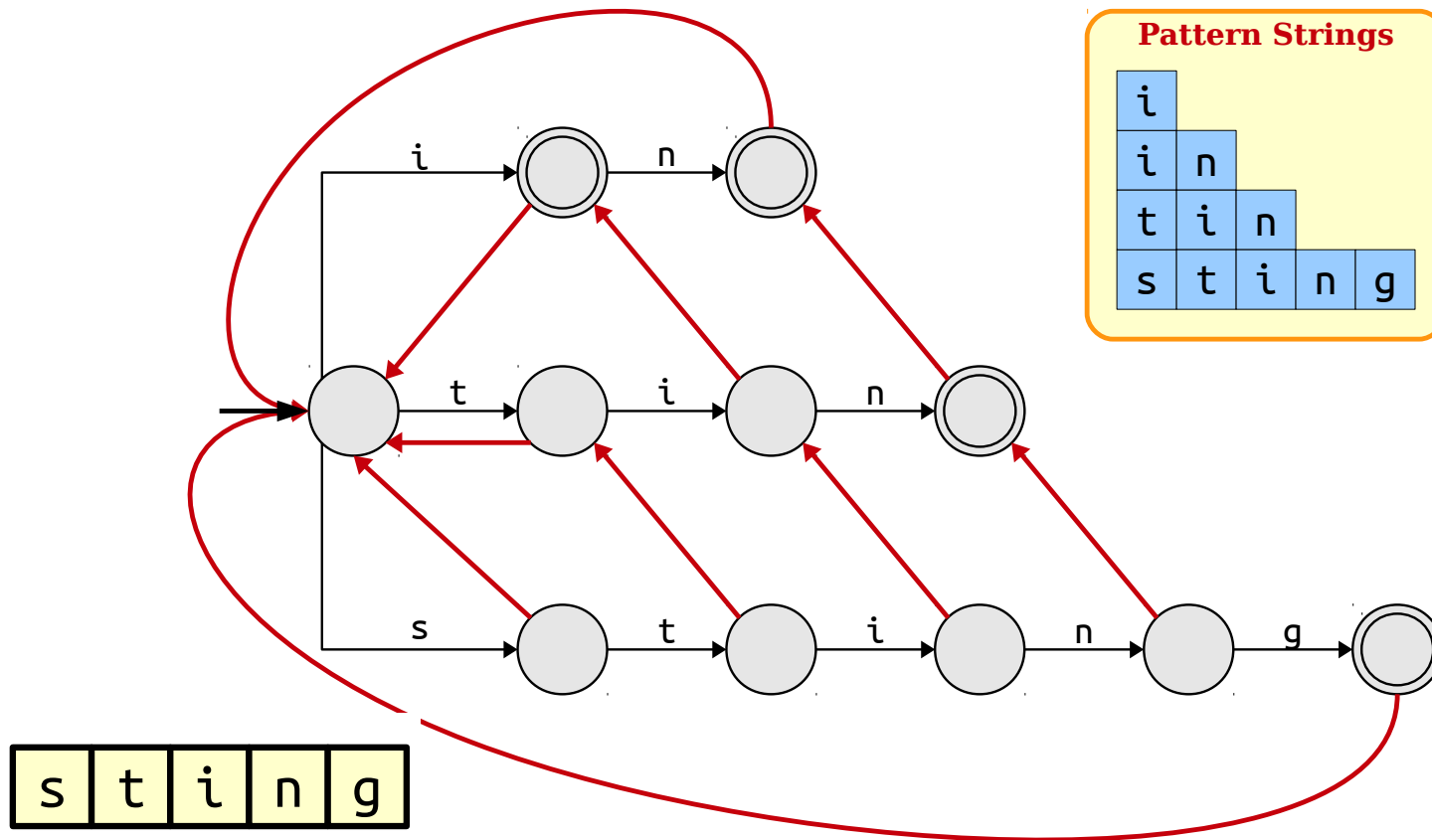


s o t a t

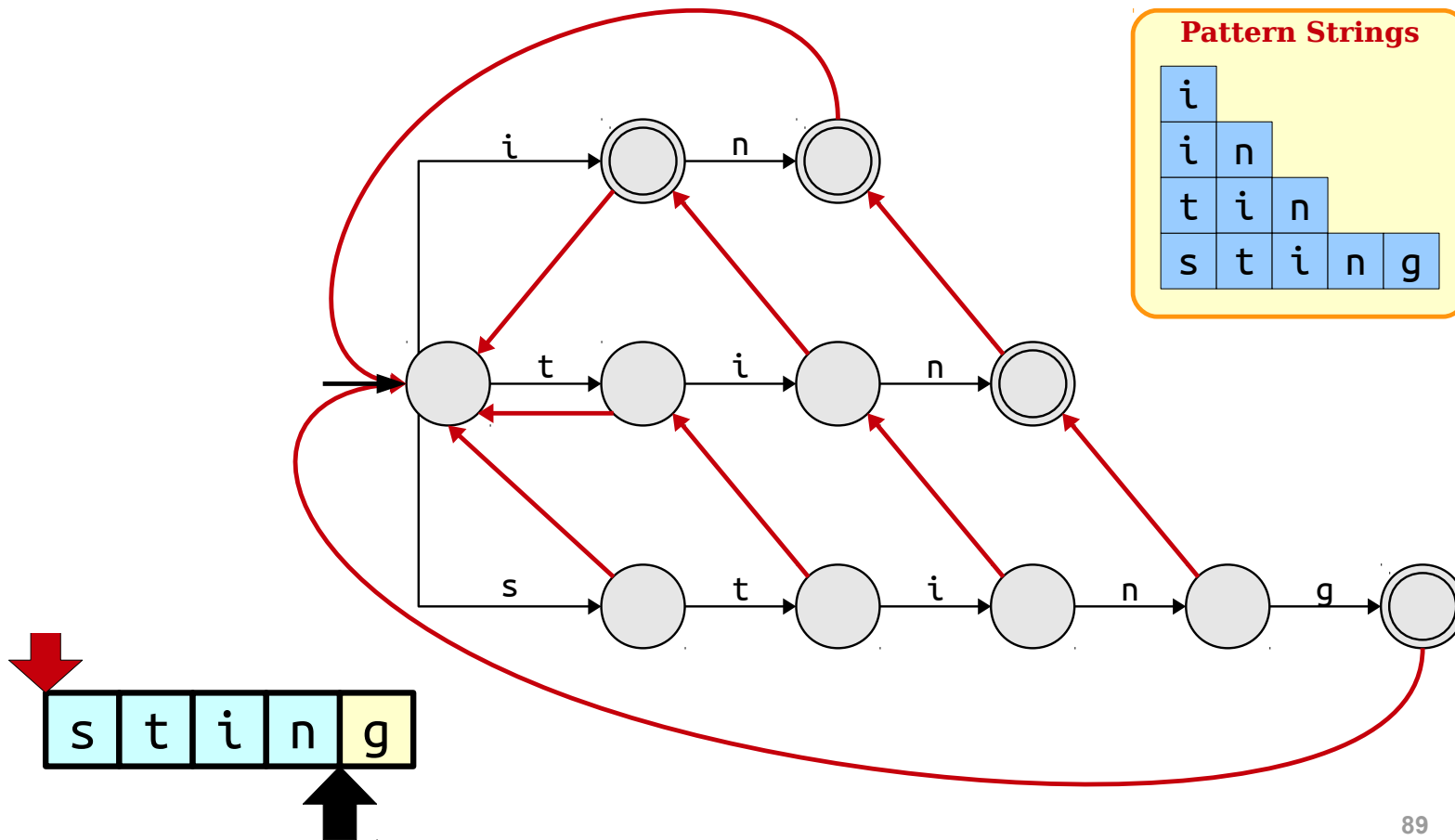


Red link is a suffix link

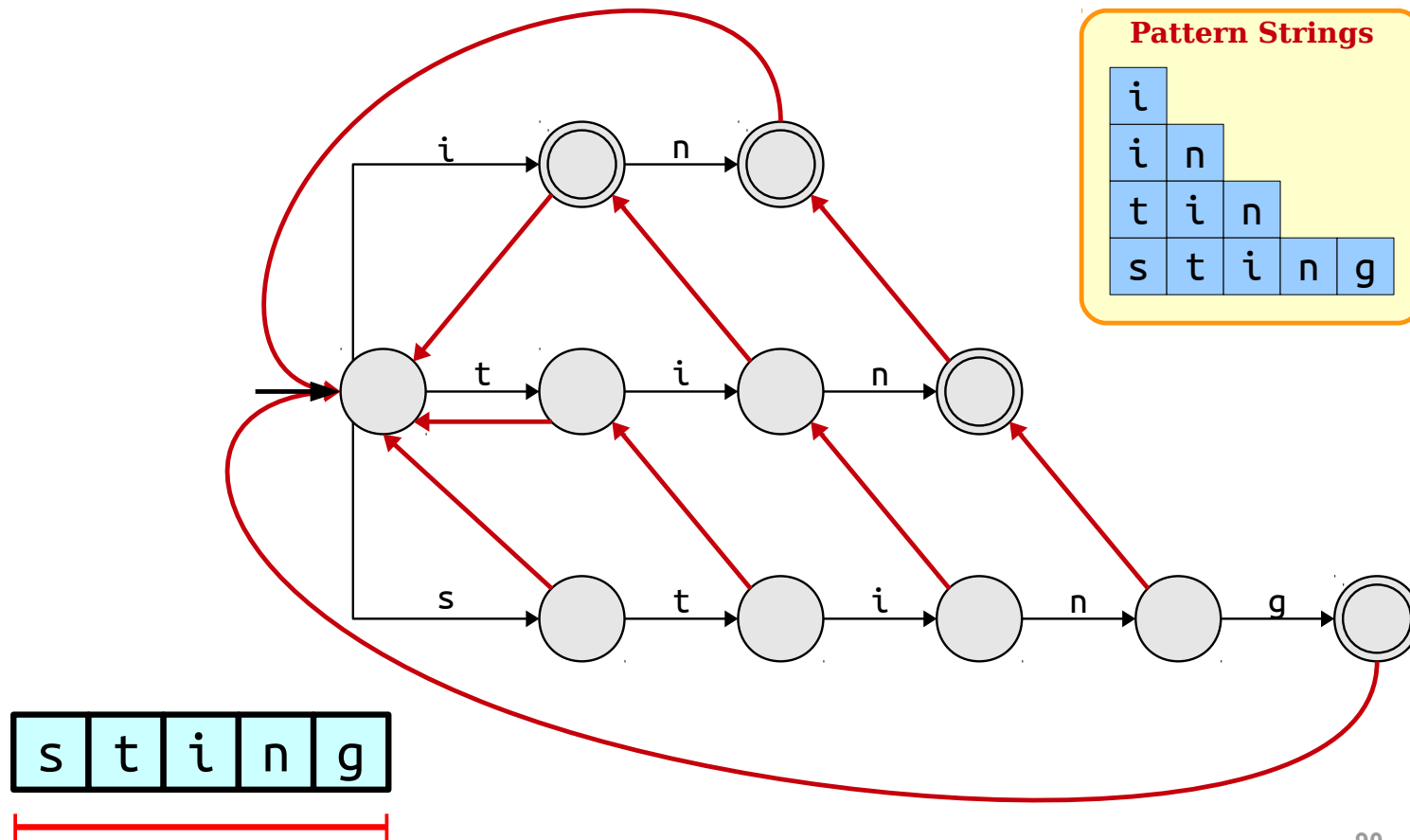
A Possible Problem: Detecting words inside other words



A Possible Problem

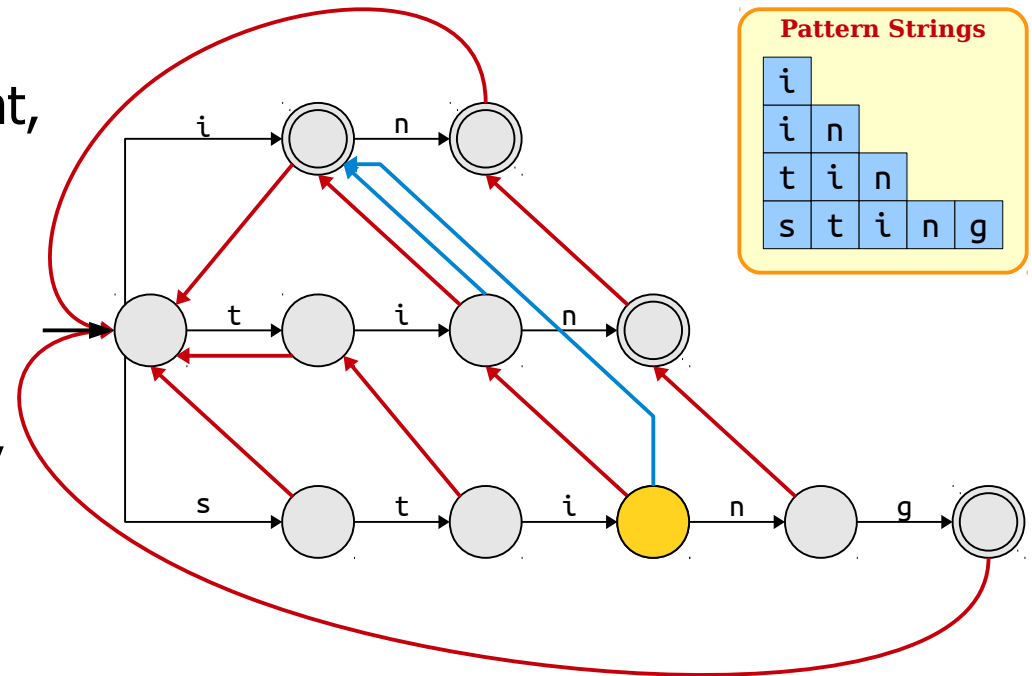


A Possible Problem

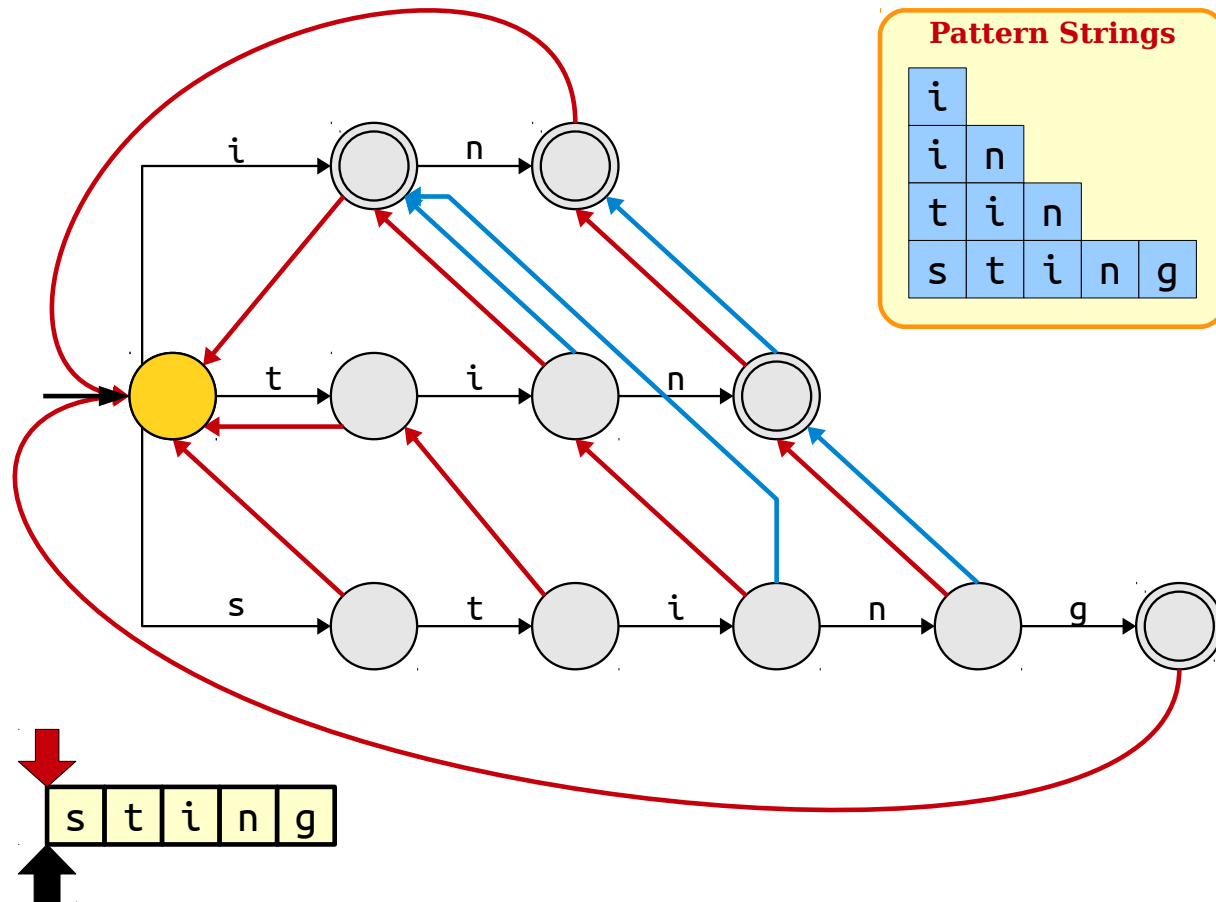


Problem?

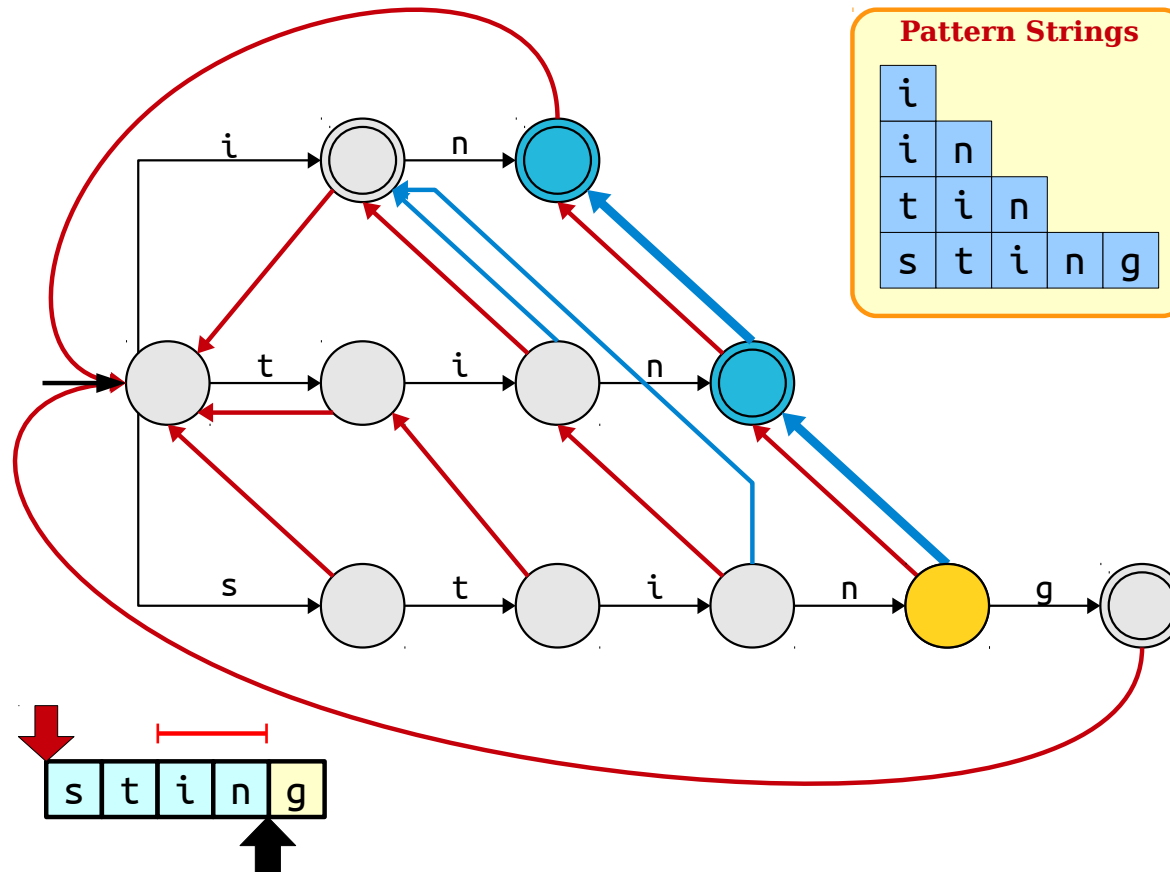
- We found “sting”, but need to find patterns inside of “sting”
 - Need to figure out how to output “i”, “in”, “tin” while searching
 - Solution: Output links (blue links!)
 - Whenever output links are present, output string pointed to output link when visiting a node
 - Precomputing where we eventually need to end up, we can read off any extra patterns we find
 - Can be done quickly!
-
- ```
graph LR; Root(()) -- i --> N1(()); Root -- t --> N2(()); Root -- s --> N3(()); N1 -- n --> End1((()); N2 -- i --> N4(()); N4 -- n --> End2((()); N3 -- t --> N5(()); N5 -- i --> End3(()); N1 -->|blue| N4; N2 -->|blue| N5; N3 -->|blue| N5; End3 -->|red| Root;
```



# Output Links



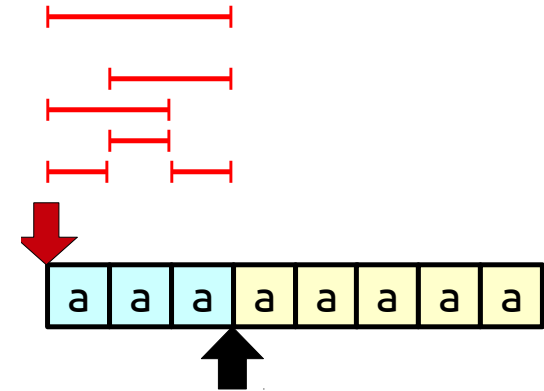
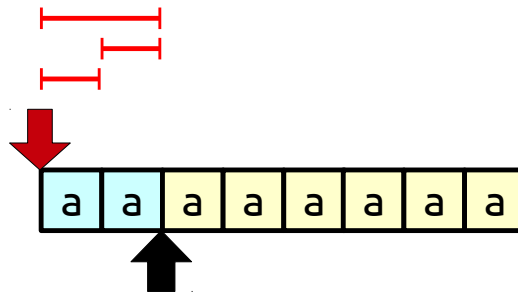
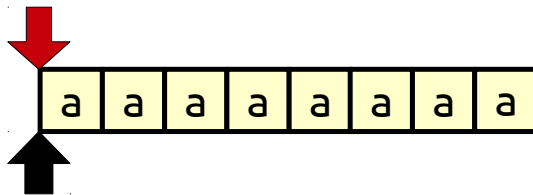
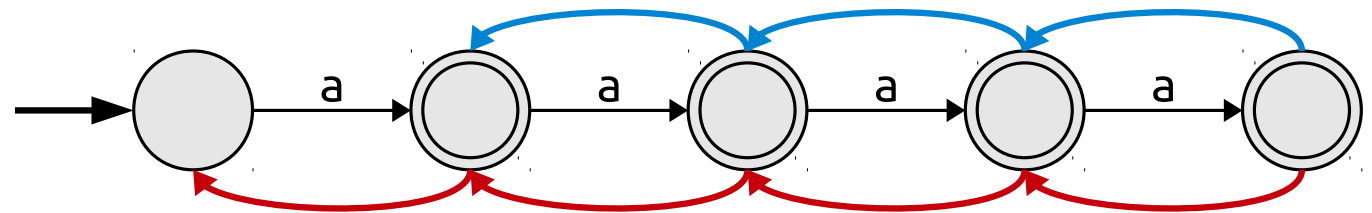
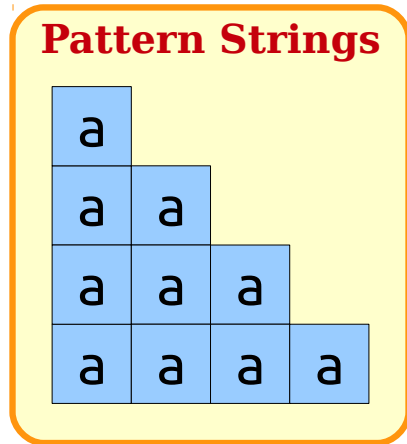
# Output Links



## Final Algorithm

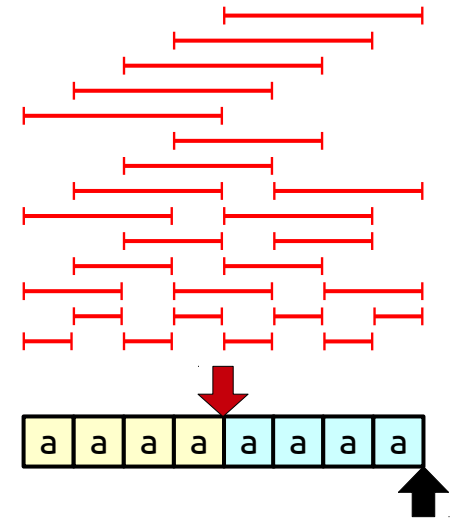
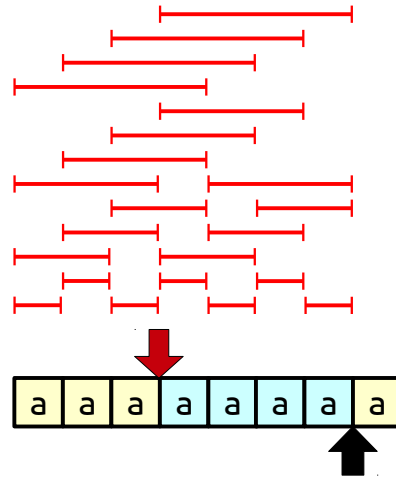
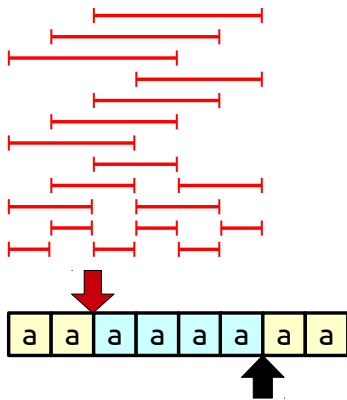
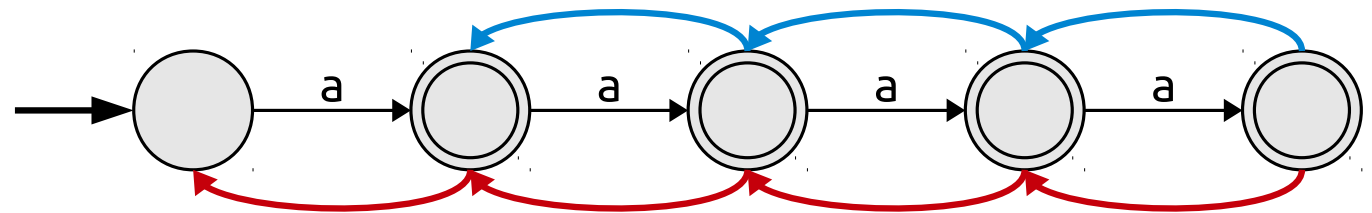
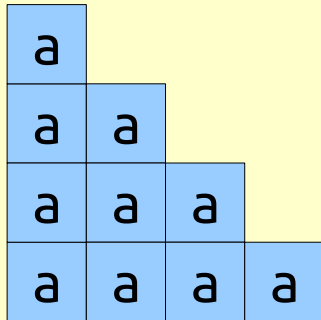
- Start at the root node in the trie
- For each character  $c$  in the string:
  - While there is no edge labeled  $c$ :
    - If you are at root, break out of loop; otherwise, follow suffix link
  - If there is an edge labeled  $c$ , follow it
    - If the current node corresponds to a pattern, output that pattern
    - Output all words in the chain of output links originating at this node
- How bad can this be? How much backtracking?

## Bad Case



## Bad Case

### Pattern Strings





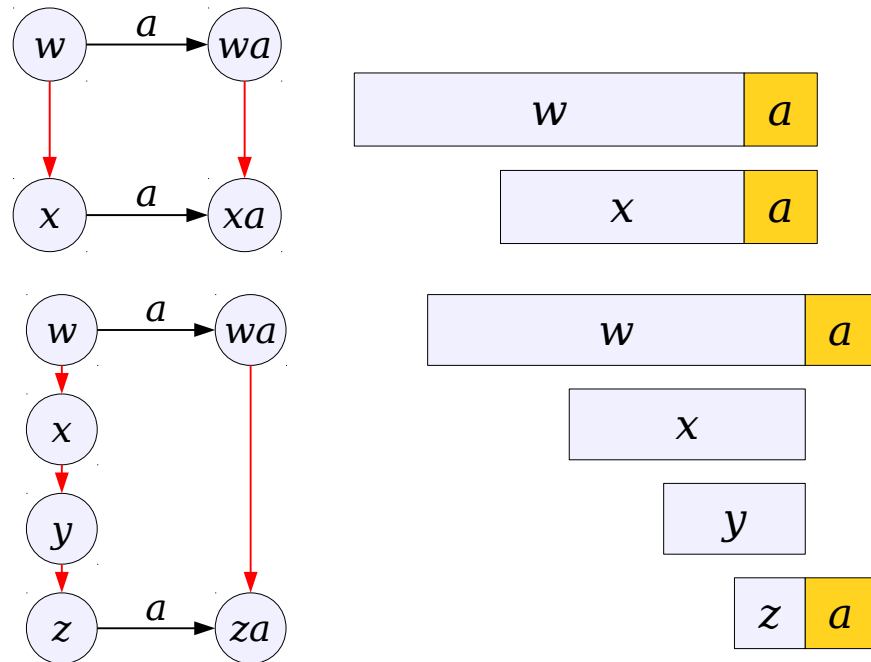
## Run Time Analysis

- This is known as the Aho-Corasick algorithm, or an Aho-Corasick automata
  - May spend a lot of time outputting matches
  - That is a function of the problem, not the algorithm
  - Any algorithm that matches strings would have to spend the time reporting matches
- Let  $z$  denote the number of matches reported by our algorithm.
  - Runtime of match phase is  $\Theta(m + z)$
- Only issue: How long does it take to build a trie, with suffix links and output links?
  - Building a trie is  $\Theta(n)$ , so need to add suffix links and output links

# Fast Suffix Link Construction

- Key insight: Suppose we know the suffix link for a node labeled  $w$ . After following a trie edge labeled  $a$ , there are two possibilities.

- Case 1:  $xa$  exists

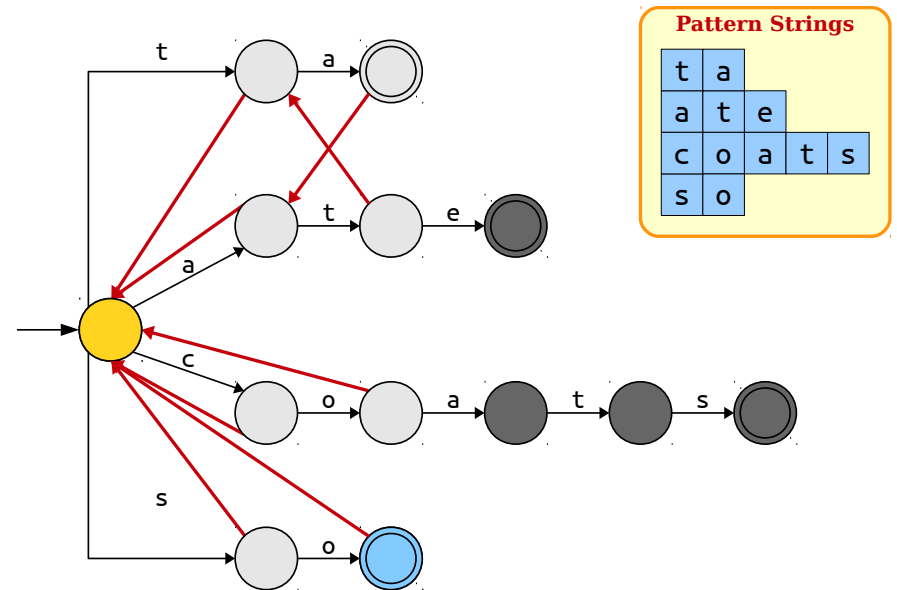
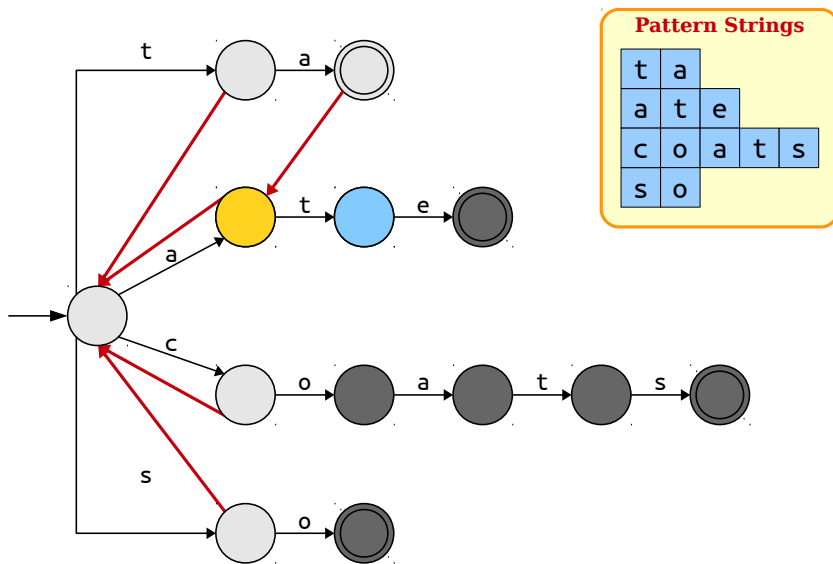


- Case 2:  $xa$  does not exist

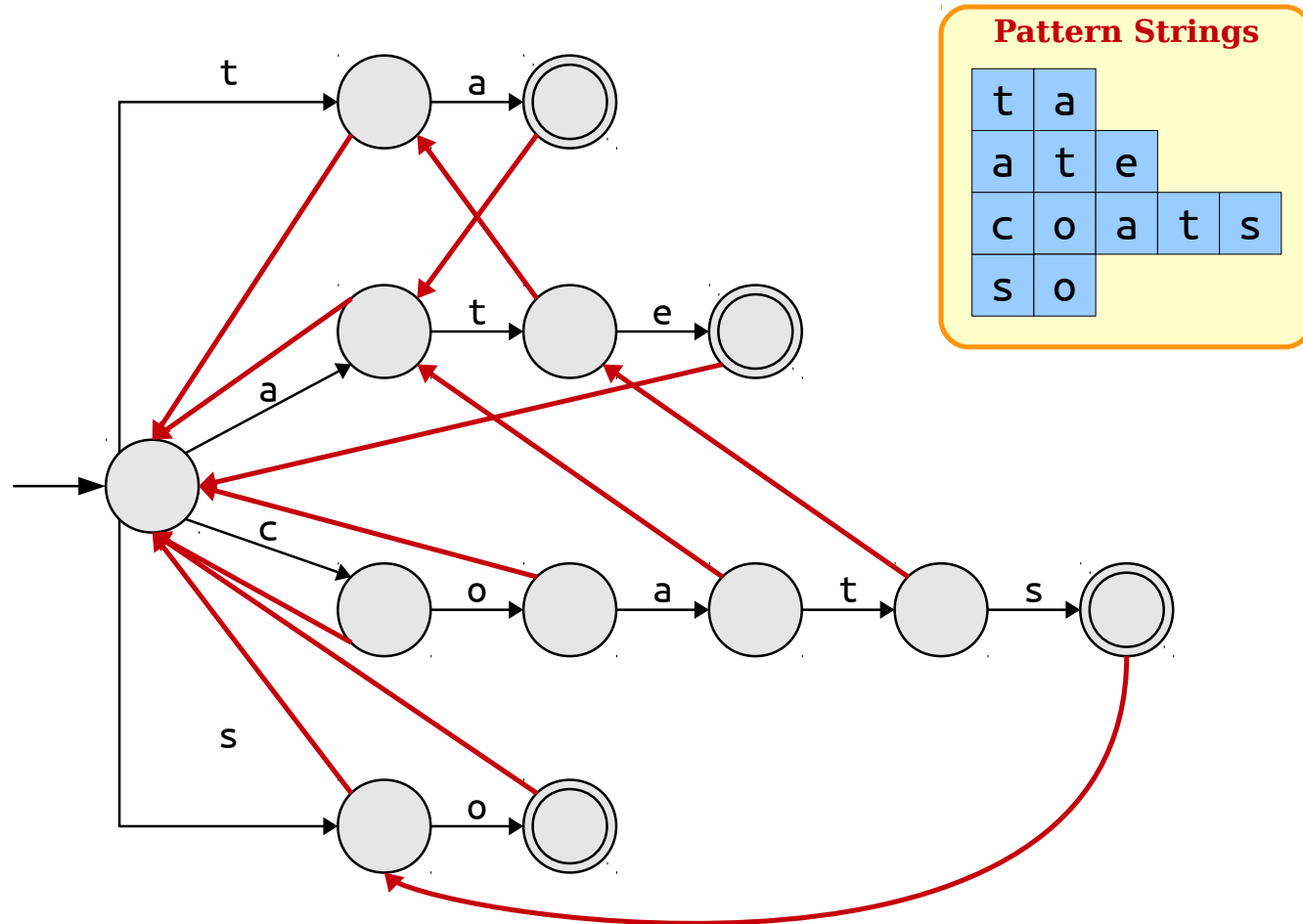
## Fast Suffix Link Construction

- To construct the suffix link for a node  $w_a$ :
  - Follow  $w$ 's suffix link to node  $x$
  - If node  $x_a$  exists,  $w_a$  has a suffix link to  $x_a$
  - Otherwise, follow  $x$ 's suffix link and repeat
  - If you need to follow backwards from the root, then  $w_a$ 's suffix link points to the root
- Observations
  - Suffix links point from longer strings to shorter strings
  - Precomputing suffix links for nodes in **ascending order** of string length, information needed will be available at the time we need it
  - Breadth first search!

# Growing the Trie



# Growing the Trie



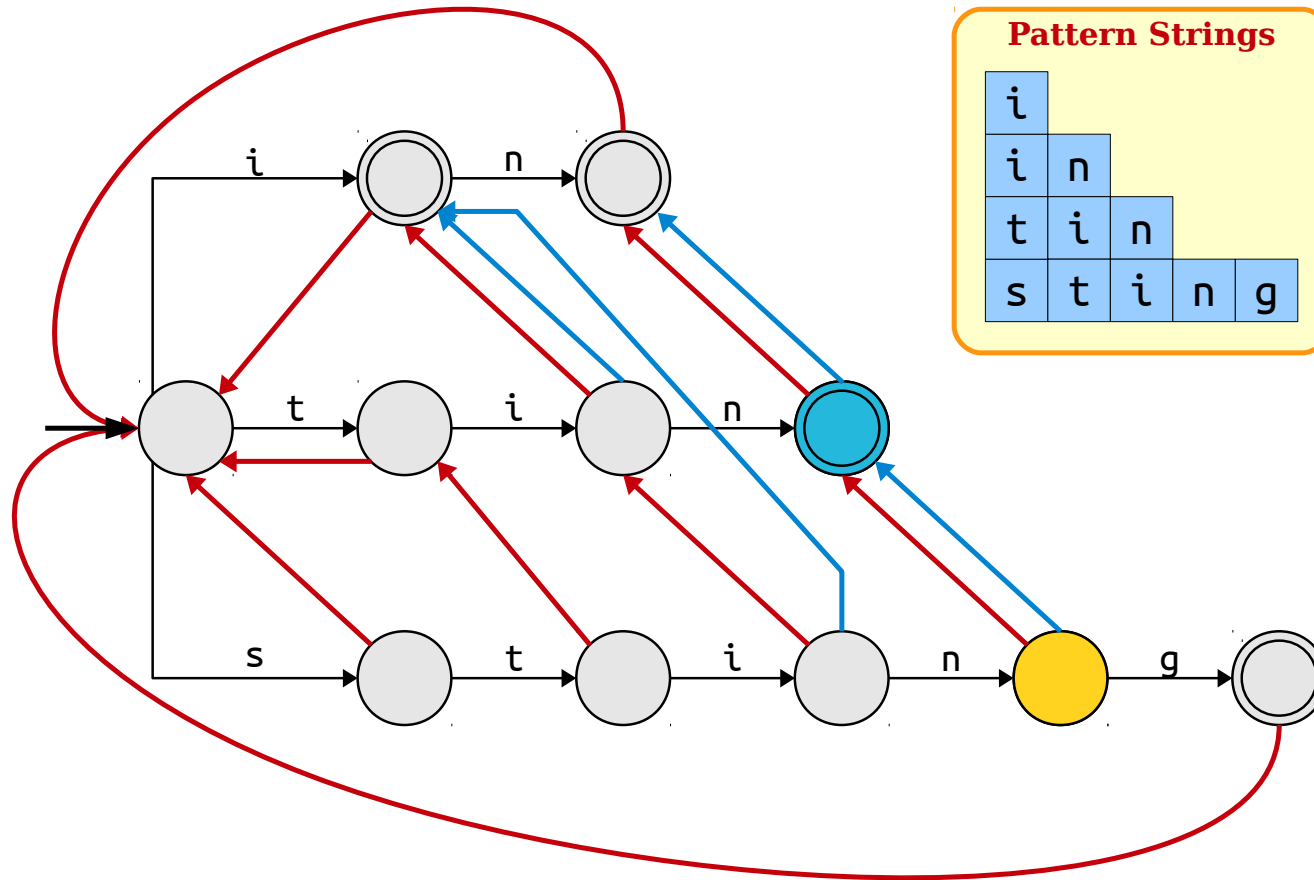
## Fast Suffix Link Algorithm

- Do a breadth-first search of the trie, performing the following operations:
  - If the node is the root, it has no suffix link
  - If the node is one hop away from the root, its suffix link points to the root
  - Otherwise, the node corresponds to some string  $wa$ . Let  $x$  be the node pointed at by  $w$ 's suffix link. Then, do the following:
    - If the node  $xa$  exists,  $wa$ 's suffix link points to  $xa$
    - Otherwise, if  $x$  is the root node,  $wa$ 's suffix link points to the root
    - Otherwise, set  $x$  to the node pointed at by  $x$ 's suffix link and repeat
- Can be done in  $O(n)$  !
  - Total time required to construct suffix links for a pattern of length  $h$ :  $O(h)$

## Output Links

- The output link at a node corresponding to a string  $w$  points to:
  - the node corresponding to the longest proper suffix of  $w$  that is a pattern, or
  - null if no such suffix exists
- By always pointing to the node corresponding to the longest suffix, we ensure that we chain together all the patterns using output links
  - Need to chain output links looking for smaller suffixes...

# Chained Output Links

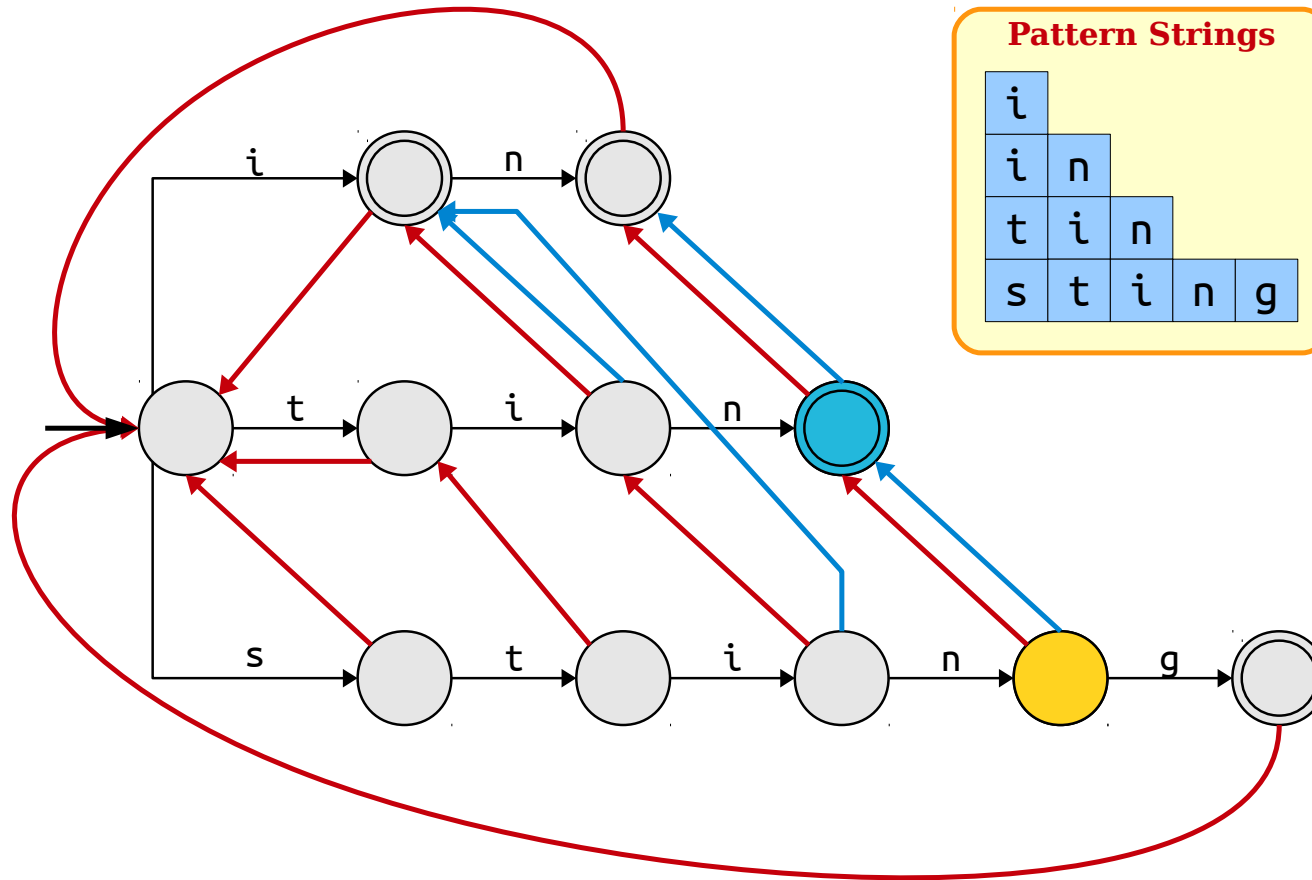




## Computing Output Links

- Initially, set every node's output link to be a null pointer
- While doing the BFS to fill in suffix links, set the output link of the current node  $v$  as follows:
  - Let  $u$  be the node pointed at by  $v$ 's suffix link
  - If  $u$  corresponds to a pattern, set  $v$ 's output link to  $u$
  - Otherwise, set  $v$ 's output link to  $u$ 's output link; this may be null...
- Can be done in  $O(n)$  !
  - Exploit Breadth-first search order

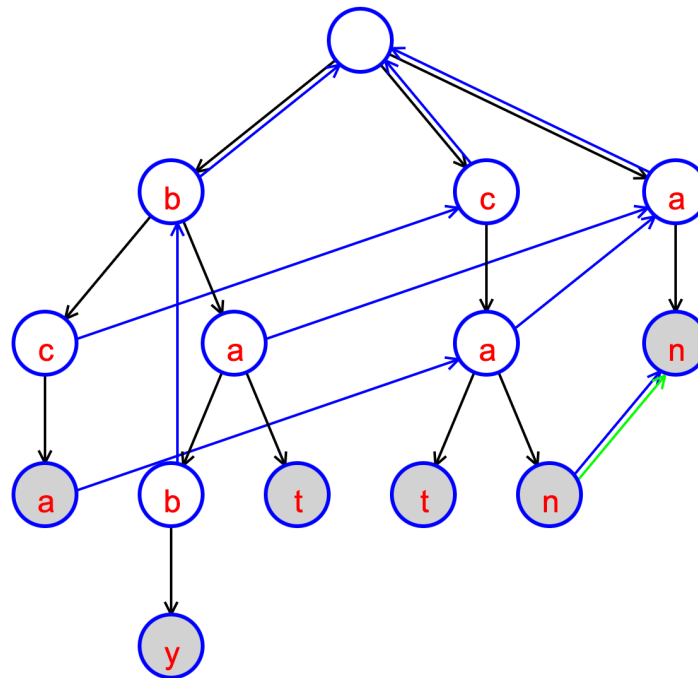
# Building Output Links



# Illustration of Aho-Corasick Algorithm

- <https://wiomoc.github.io/aho-corasick-viz/>

baby cat can run at a bat



|      |   |
|------|---|
|      | + |
| bca  | × |
| baby | × |
| cat  | × |
| can  | × |
| an   | × |
| bat  | × |

## Summary: Aho-Corasick Algorithm

- Finds  $k$  patterns with total length  $n$  in text of length  $m$  by building a suffix trie in  $O(n)$  and finding  $z$  matches in  $O(m+z)$ 
  - Generalizes KMP algorithm
  - Exploits trie structures and prefix backtracking
- Lots of work continues in this area
  - Driven by computer search of text sources
- Most language searches for regular expressions are slow (use simple methods)
  - But fast methods exist